

Departamento de Informática

Acceso a Datos: Spring Rest y Mongo en acción

Juan Gualberto Gutiérrez Marín

Enero 2026

AVISO: El texto contiene muchos enlaces embebidos que sólo funcionan si se abre el PDF desde un lector digital. Si se imprime, se perderá mucha información interesante. No obstante, en el apartado “Bibliografía” se han añadido los enlaces para aquellos lectores que deseen una copia impresa y puedan ver la información.

Este documento se encuentra bajo una licencia Creative Commons de Atribución-CompartirIgual (CC BY-SA).

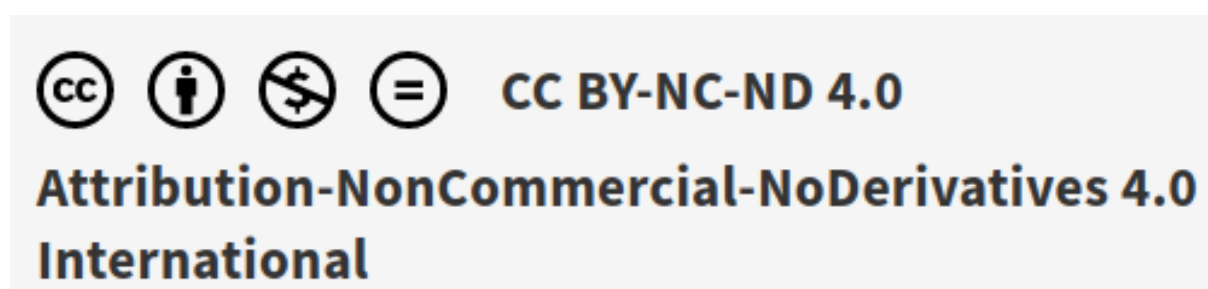


Figura 1: Atribución-CompartirIgual (CC BY-SA)

Esto significa que puedes:

- Compartir: copiar y redistribuir el material en cualquier medio o formato.
- Adaptar: remezclar, transformar y construir sobre el material para cualquier propósito, incluso comercialmente.

Bajo las siguientes condiciones:

- Atribución: debes dar crédito de manera adecuada, proporcionar un enlace a la licencia e indicar si se han realizado cambios. Puedes hacerlo de cualquier manera razonable, pero no de una manera que sugiera que el licenciante te respalda a ti o al uso que hagas del trabajo.
- Compartir igual: si remezclas, transformas o creas a partir del material, debes distribuir tus contribuciones bajo la misma licencia que el original.

Para más detalles, consulta la licencia completa.

Para una versión actualizada de este libro visita esta Web: <https://gitlab.iesvirgendelcarmen.com/juangu>.

Índice

1	Introducción a las bases de datos NoSQL	6
1.1	Introducción a las bases de datos NoSQL	6
1.1.1	Características generales de las bases de datos NoSQL	6
1.1.2	Tipos principales de bases de datos NoSQL	6
1.1.3	Ventajas e inconvenientes de NoSQL frente a bases de datos relacionales	8
1.2	MongoDB	10
1.2.1	Conceptos fundamentales	10
1.2.2	Instalación y configuración de MongoDB	10
1.2.3	Conexión interactiva a MongoDB	13
1.2.4	Operaciones CRUD básicas en MongoDB	15
1.2.5	Colección de ejemplo: estudiantes	16
1.3	Introducción: REST + MongoDB + SPA ligera	18
1.3.1	Objetivo general del capítulo	18
1.3.2	Arquitectura de la aplicación	18
1.3.3	Por qué REST, MongoDB y una SPA ligera	19
1.3.4	Caso práctico: gestión de reservas de pistas deportivas	19
1.4	Preparación del entorno y dependencias	23
1.4.1	Requisitos previos	23
1.4.2	Estructura de carpetas recomendada	23
1.4.3	Creación del archivo <code>.env</code> (variables compartidas)	23
1.4.4	Preparación del Dev Container	24
1.4.5	Creación del proyecto Spring Boot (Maven)	26
1.4.6	Dependencias (starters) necesarias	27
1.4.7	Configuración en YAML (<code>application.yml</code>) usando variables de entorno	27
1.4.8	Verificación del entorno	28
1.4.9	Nota sobre “mongoengine”	29
1.4.10	Archivo <code>.env</code> de ejemplo	29
1.4.11	Archivo <code>.gitignore</code> recomendado	30
1.4.12	Arranque de la aplicación Spring Boot desde el contenedor	30
1.4.13	Cheatsheet para troubleshooting	31
1.5	MongoDB: documentos, colecciones y ObjectId	33
1.5.1	Documentos	33
1.5.2	Colecciones	33
1.5.3	ObjectId	34
1.5.4	Documentos embebidos y referencias	34
1.5.5	Comparación con el modelo relacional	35

1.6	Spring Data MongoDB: @Document, repositorios y consultas	37
1.6.1	Dependencias necesarias	37
1.6.2	Diseño del modelo y decisiones de persistencia	37
1.6.3	Paquetes recomendados	37
1.6.4	1) Documentos y embebidos	38
1.6.5	2) Repositorios (MongoRepository) y consultas	43
1.6.6	Consideraciones de consulta y modelado	44
1.6.7	Estado del proyecto tras este apartado	45
1.7	Diseño de la API REST: endpoints, DTOs, códigos HTTP	46
1.7.1	Principios de diseño aplicados	46
1.7.2	Recursos y endpoints	46
1.7.3	DTOs (Data Transfer Objects)	49
1.7.4	Códigos HTTP y comportamiento esperado	50
1.7.5	Ejemplos de intercambio (contrato)	51
1.7.6	Preparación para el siguiente apartado	51
1.8	Validación, DTOs y Control de Errores (@ControllerAdvice)	52
1.8.1	1. Entendiendo el patrón DTO (Data Transfer Object)	52
1.8.2	2. Implementación de Validación en DTOs	53
1.8.3	3. Manejo Centralizado de Excepciones	56
1.8.4	4. Resumen de Códigos de Estado HTTP	61
1.8.5	Conclusiones	62
1.9	Validación y control de errores (@ControllerAdvice)	63
1.9.1	1) Validación de entrada en la API	63
1.9.2	2) DTOs con anotaciones de Bean Validation	64
1.9.3	3) Modelo de error unificado	65
1.9.4	4) Excepciones de dominio	65
1.9.5	5) Manejo centralizado con @ControllerAdvice	66
1.9.6	6) Reglas de negocio y códigos HTTP	67
1.9.7	Resultado de este apartado	68
1.10	CORS y configuración por entornos	69
1.10.1	1) Conceptos básicos de CORS	69
1.10.2	2) Estrategias posibles en este proyecto	69
1.10.3	3) Configuración de CORS en Spring Boot	70
1.10.4	4) Configuración por entornos con perfiles	71
1.10.5	5) Variables de entorno necesarias	72
1.10.6	6) Verificación de CORS	72
1.10.7	Resultado de este apartado	73

1.11	Front con Bootstrap: estructura y componentes	74
1.11.1	Estructura de archivos recomendada	74
1.11.2	Diseño de la interfaz	74
1.11.3	index.html (estructura base)	75
1.11.4	css/app.css (opcional)	79
1.11.5	Plantillas de tabla (contrato de renderizado)	80
1.11.6	Objetivo técnico del apartado	80
1.11.7	Consumo de API desde JS (jQuery)	82
2	Bibliografía	92

1 Introducción a las bases de datos NoSQL

1.1 Introducción a las bases de datos NoSQL

Las bases de datos **NoSQL (Not Only SQL)** surgen como respuesta a las limitaciones de las bases de datos relacionales tradicionales en determinados escenarios, especialmente aquellos relacionados con **gran volumen de datos, alta concurrencia, escalabilidad horizontal** y **modelos de datos cambiantes**.

A diferencia de los sistemas relacionales, NoSQL no impone un modelo único basado en tablas, filas y relaciones estrictas, sino que ofrece distintos enfoques de almacenamiento adaptados a diferentes necesidades.

1.1.1 Características generales de las bases de datos NoSQL

Las bases de datos NoSQL comparten una serie de características comunes:

- **Esquema flexible:** los datos no necesitan ajustarse a una estructura rígida predefinida.
- **Escalabilidad horizontal:** están diseñadas para escalar añadiendo nodos, no aumentando la potencia de uno solo.
- **Alto rendimiento** en operaciones de lectura y escritura.
- **Modelo de consistencia flexible**, habitualmente orientado a consistencia eventual en lugar de transacciones ACID estrictas.
- **Orientación al dominio:** el modelo de datos suele adaptarse a cómo se consume la información.

Estas características las hacen especialmente adecuadas para aplicaciones web modernas, sistemas distribuidos y arquitecturas basadas en microservicios.

1.1.2 Tipos principales de bases de datos NoSQL

Las bases de datos NoSQL se clasifican habitualmente en cuatro grandes categorías:

1.1.2.1 Bases de datos clave-valor Almacenan datos como pares `clave` $\rightarrow$ `valor`.

Ejemplo conceptual:

```
1 "session_1234"  $\rightarrow$  { "usuario": "ana", "rol": "admin" }
```

Uso típico:

- Cachés
- Sesiones de usuario
- Configuración temporal

Ventajas:

- Simplicidad
- Velocidad extrema

Inconvenientes:

- Consultas muy limitadas
- No aptas para datos complejos

1.1.2.2 Bases de datos orientadas a columnas Organizan los datos por columnas en lugar de por filas.

Ejemplo conceptual:

```
1 usuario:1:nombre $-> "Luis"
2 usuario:1:email  $-> "luis@email.com"
```

Uso típico:

- Analítica
- Big Data
- Series temporales

Ventajas:

- Excelente rendimiento en grandes volúmenes
- Compresión eficiente

Inconvenientes:

- Curva de aprendizaje
- No pensadas para CRUD clásico

1.1.2.3 Bases de datos orientadas a grafos Modelan los datos como **nodos** y **relaciones**.

Ejemplo conceptual:

```
1 (Usuario)-[RESERVA]->(Pista)
```


Uso típico:

- Redes sociales
- Sistemas de recomendación
- Análisis de relaciones complejas

Ventajas:

- Consultas relacionales muy potentes
- Modelo muy expresivo

Inconvenientes:

- No adecuadas para consultas simples masivas
- Modelo menos intuitivo para CRUD básicos

1.1.2.4 Bases de datos orientadas a documentos Almacenan la información en forma de **documentos estructurados**, normalmente en formato JSON o BSON.

Ejemplo conceptual:

```
1 {  
2   "nombre": "Pista Central",  
3   "ciudad": "Jaén",  
4   "tipo": "Tenis"  
5 }
```

Este tipo de base de datos es el que se utilizará en este capítulo.

1.1.3 Ventajas e inconvenientes de NoSQL frente a bases de datos relacionales

Ventajas

- Mayor flexibilidad del modelo de datos.
- Menor acoplamiento entre aplicación y estructura de almacenamiento.
- Escalabilidad sencilla.
- Mejor adaptación a dominios complejos.

Inconvenientes

- Menor estandarización.
- Ausencia de JOINS clásicos.
- Transacciones limitadas o no disponibles según el motor.
- Mayor responsabilidad en la lógica de aplicación.

Estas diferencias hacen que la elección entre SQL y NoSQL dependa del **caso de uso**, no de una superioridad técnica general.

1.2 MongoDB

MongoDB es un ejemplo de base de datos documental. Las **bases de datos documentales** almacenan la información en forma de documentos completos y autocontenidos. Cada documento representa una entidad del dominio y puede incluir estructuras anidadas, listas y subdocumentos.

Este enfoque se alinea de forma natural con los objetos utilizados en los lenguajes de programación orientados a objetos y facilita el desarrollo de aplicaciones backend modernas.

1.2.1 Conceptos fundamentales

1.2.1.1 Documento Unidad básica de almacenamiento. Equivale conceptualmente a una fila en una base de datos relacional, pero con estructura jerárquica.

Ejemplo:

```
1 {  
2   "_id": "64f8a2...",  
3   "nombre": "Instalación Norte",  
4   "ciudad": "Jaén"  
5 }
```

1.2.1.2 Colección Conjunto de documentos del mismo tipo lógico. Equivale a una tabla.

Ejemplo:

- usuarios
- instalaciones
- reservas

1.2.1.3 Objectid Identificador único generado automáticamente por MongoDB. Incluye información temporal y garantiza unicidad sin coordinación central.

1.2.2 Instalación y configuración de MongoDB

Para crear la infraestructura de servicios necesaria, en lugar de instalar MongoDB directamente en el sistema operativo, y siguiendo la metodología habitual del módulo, se utilizarán **contenedores Docker**. En concreto, se desplegarán dos servicios:

- Un contenedor para el **servidor MongoDB**, que actuará como base de datos documental.

- Un contenedor para **Mongo Express**, una interfaz web que permite inspeccionar y administrar MongoDB de forma visual.

Este enfoque presenta varias ventajas: homogeneiza el entorno de trabajo, reduce el tiempo de configuración inicial y evita dependencias con el sistema anfitrión.

Para ello, se crea una carpeta llamada `stack` y, dentro de ella, el archivo `docker-compose.yml` con el siguiente contenido:

```
1 version: '3.1'
2
3 services:
4
5   mongo:
6     image: mongo
7     restart: 'no'
8     environment:
9       MONGO_INITDB_ROOT_USERNAME: root
10      MONGO_INITDB_ROOT_PASSWORD: 83uddjfp0cmMD
11     ports:
12       - 27017:27017
13
14   mongo-express:
15     image: mongo-express
16     restart: 'no'
17     ports:
18       - 8081:8081
19     environment:
20       ME_CONFIG_BASICAUTH_USERNAME: mongo
21       ME_CONFIG_BASICAUTH_PASSWORD: 83uddjfp0cmMD
22       ME_CONFIG_MONGODB_ADMINUSERNAME: root
23       ME_CONFIG_MONGODB_ADMINPASSWORD: 83uddjfp0cmMD
24       ME_CONFIG_MONGODB_URL: mongodb://root:83uddjfp0cmMD@mongo:27017/
```

1.2.2.1 docker-compose.yml El archivo `docker-compose.yml` permite definir y ejecutar múltiples servicios Docker de forma coordinada. A continuación se analizan sus principales secciones.

```
1 version: '3.1'
```

Esta línea indica la versión del esquema de Docker Compose utilizada. La versión 3.1 es compatible con la mayoría de instalaciones actuales de Docker.

1.2.2.1.1 Servicio mongo

```
1 mongo:
2   image: mongo
3   restart: 'no'
```

```
4  environment:
5    MONGO_INITDB_ROOT_USERNAME: root
6    MONGO_INITDB_ROOT_PASSWORD: 83uddjfp0cmMD
7  ports:
8    - 27017:27017
```

Este servicio define una instancia de MongoDB basada en la imagen oficial:

- **image:** se utiliza la imagen oficial `mongo` desde Docker Hub.
- **restart: 'no':** el contenedor no se reiniciará automáticamente si se detiene.
- **environment:**
 - Se define un usuario administrador (`root`) y su contraseña.
 - Estas credenciales se utilizan para acceder a la base de datos.
- **ports:**
 - Se expone el puerto estándar de MongoDB (27017), permitiendo la conexión desde el host y desde otros contenedores.

1.2.2.2 Servicio mongo-express

```
1  mongo-express:
2    image: mongo-express
3    restart: 'no'
4    ports:
5      - 8081:8081
6    environment:
7      ME_CONFIG_BASICAUTH_USERNAME: mongo
8      ME_CONFIG_BASICAUTH_PASSWORD: 83uddjfp0cmMD
9      ME_CONFIG_MONGODB_ADMINUSERNAME: root
10     ME_CONFIG_MONGODB_ADMINPASSWORD: 83uddjfp0cmMD
11     ME_CONFIG_MONGODB_URL: mongodb://root:83uddjfp0cmMD@mongo:27017/
```

Este servicio despliega Mongo Express, una interfaz web de administración:

- **ports:**
 - Se expone el puerto 8081, que permite acceder a la interfaz desde el navegador.
- **ME_CONFIG_BASICAUTH_USERNAME / PASSWORD:**
 - Definen las credenciales para acceder a la interfaz web.
- **ME_CONFIG_MONGODB_ADMINUSERNAME / PASSWORD:**
 - Son las credenciales con las que Mongo Express se autentica contra MongoDB.

- **ME_CONFIG_MONGODB_URL:**

- Cadena de conexión a MongoDB, donde se especifica el usuario, contraseña y nombre del servicio ([mongo](#)).

Es importante diferenciar claramente entre:

- **MongoDB:** motor de base de datos documental.
- **Mongo Express:** herramienta web de administración.

Aunque este entorno es de desarrollo, **siempre se deben usar contraseñas robustas**, ya que una exposición accidental del servicio podría comprometer los datos almacenados.

1.2.2.3 Puesta en marcha de los servicios Una vez creado el archivo `docker-compose.yml`, y teniendo Docker y Docker Compose instalados, se ejecuta el siguiente comando desde la carpeta `stack`:

```
1 docker-compose up -d
```

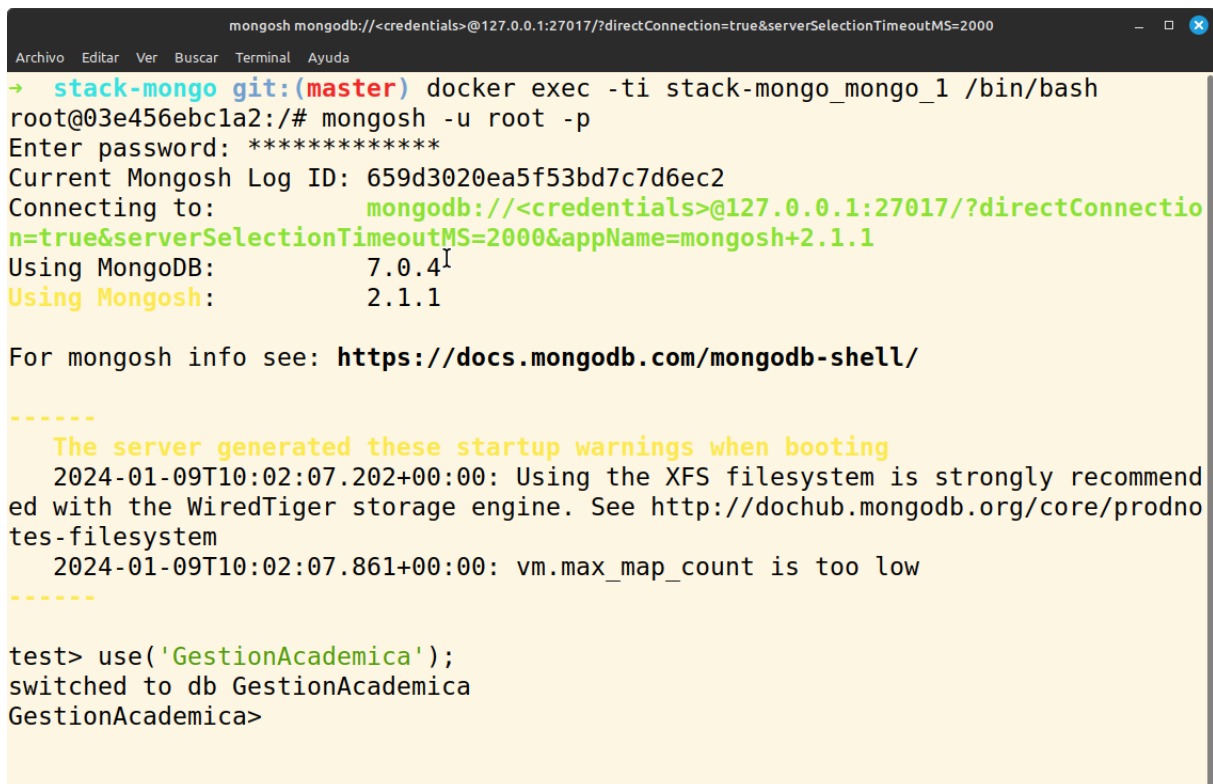
Este comando descarga las imágenes necesarias y lanza los contenedores en segundo plano. Tras unos segundos:

- MongoDB estará disponible en el puerto **27017**.
- Mongo Express será accesible desde el navegador en <http://localhost:8081>.

1.2.3 Conexión interactiva a MongoDB

Aunque en las aplicaciones reales el acceso a MongoDB se realiza desde el backend, resulta muy útil conocer el uso del **shell interactivo** para pruebas, aprendizaje y depuración.

MongoDB utiliza actualmente **mongosh**, que sustituye al antiguo [mongo](#).



```
mongosh mongodb://<credentials>@127.0.0.1:27017/?directConnection=true&serverSelectionTimeoutMS=2000
Archivo  Editar  Ver  Buscar  Terminal  Ayuda
→ stack-mongo git:(master) docker exec -ti stack-mongo_mongo_1 /bin/bash
root@03e456ebc1a2:/# mongosh -u root -p
Enter password: *****
Current Mongosh Log ID: 659d3020ea5f53bd7c7d6ec2
Connecting to:      mongodb://<credentials>@127.0.0.1:27017/?directConnectio
n=true&serverSelectionTimeoutMS=2000&appName=mongosh+2.1.1
Using MongoDB:      7.0.4
Using Mongosh:      2.1.1

For mongosh info see: https://docs.mongodb.com/mongodb-shell/

-----
  The server generated these startup warnings when booting
  2024-01-09T10:02:07.202+00:00: Using the XFS filesystem is strongly recommend
ed with the WiredTiger storage engine. See http://dochub.mongodb.org/core/prodno
tes-filesystem
  2024-01-09T10:02:07.861+00:00: vm.max_map_count is too low
-----

test> use('GestionAcademica');
switched to db GestionAcademica
GestionAcademica>
```

Figura 2: Ejemplo de conexión en modo interactivo.

1.2.3.1 Acceso al shell de MongoDB

1. Abrir una terminal dentro del contenedor MongoDB:

```
1 docker exec -ti stack-mongo_mongo_1 /bin/bash
```

2. Iniciar el shell interactivo:

```
1 mongosh -u root -p
```

Se solicitará la contraseña configurada en el `docker-compose.yml`.

3. Seleccionar una base de datos:

```
1 use('GestionAcademica')
```

Si la base de datos no existe, MongoDB la creará automáticamente cuando se inserte el primer documento.

1.2.4 Operaciones CRUD básicas en MongoDB

Vamos a suponer que tenemos el siguiente diagrama UML:

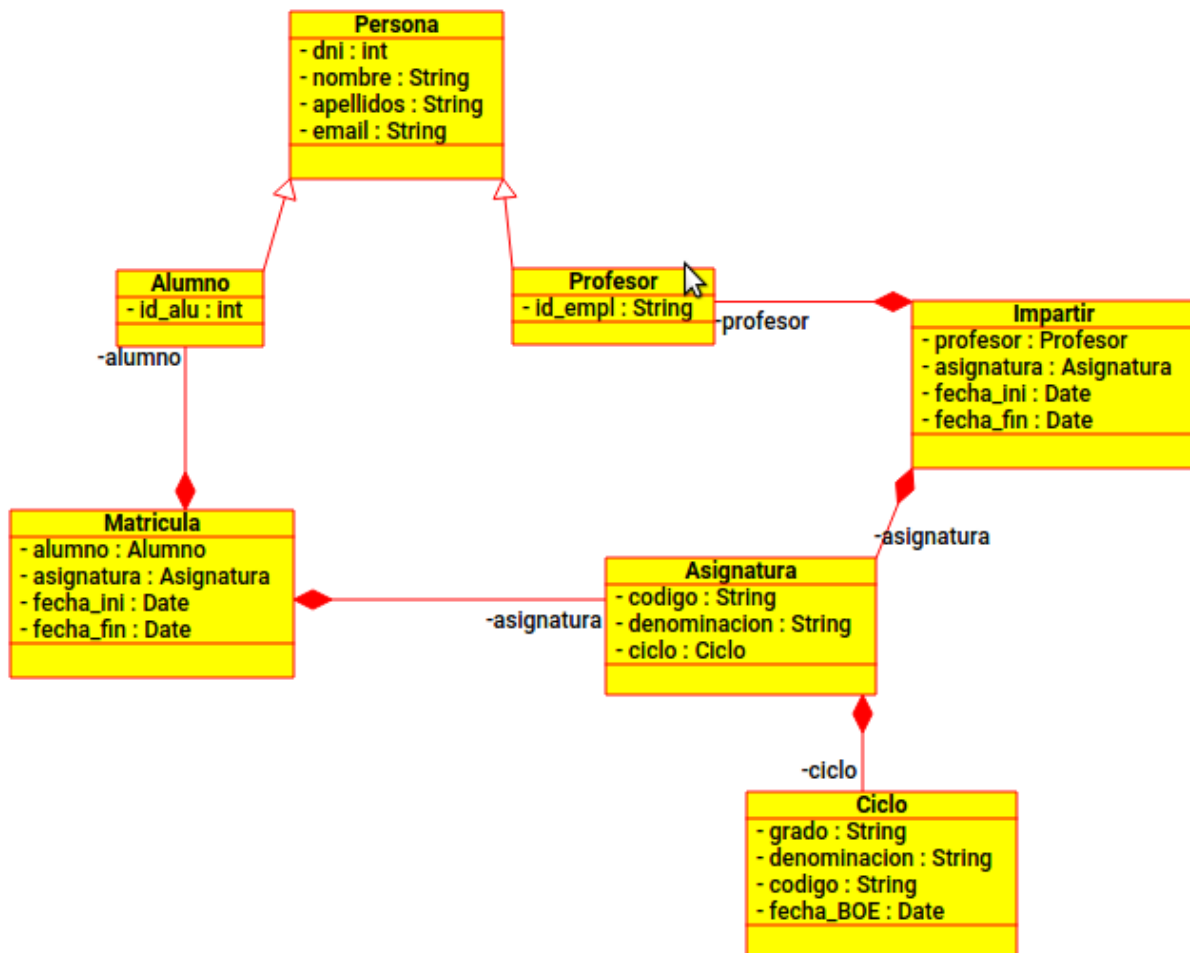


Figura 3: Diagrama UML Gestión Académica

1.2.4.1 Crear un documento (Create) Para insertar un documento en una colección llamada `alumnos`:

```

1 db.alumnos.insertOne({
2   nombre: "Juan Perez",
3   edad: 20,
4   carrera: "Ingeniería Informática"
5 })
  
```

MongoDB crea automáticamente la colección si no existe y asigna un identificador `_id`.

1.2.4.2 Leer documentos (Read) Para recuperar todos los documentos de la colección:

```
1 db.alumnos.find()
```

1.2.4.3 Actualizar un documento (Update) Para modificar un campo concreto se utiliza el operador `$set`:

```
1 db.alumnos.updateOne(  
2   { nombre: "Juan Perez" },  
3   { $set: { carrera: "Ingeniería Eléctrica" } }  
4 )
```

1.2.4.4 Eliminar un documento (Delete) Para eliminar un documento:

```
1 db.alumnos.deleteOne({ nombre: "Juan Perez" })
```

1.2.4.5 Consultas con condiciones y arrays Buscar profesores por nombre:

```
1 db.profesor.find({ nombre: "Juan" }).pretty()
```

Buscar profesores con asignaturas de más de 4 horas:

```
1 db.profesor.find(  
2   asignaturas: {  
3     $elemMatch: {  
4       horas: { $gt: 4 }  
5     }  
6   }  
7 })
```

1.2.5 Colección de ejemplo: estudiantes

Se utilizará una colección ficticia para practicar consultas:

```
1 db.estudiantes.insertMany([  
2   { nombre: "Ana García", edad: 22, carrera: "Biología", semestre: 5,  
3     promedio: 8.5 },  
4   { nombre: "Carlos López", edad: 21, carrera: "Ingeniería Civil",  
5     semestre: 4, promedio: 7.2 },  
6   { nombre: "María Torres", edad: 20, carrera: "Psicología", semestre:  
7     3, promedio: 9.0 },  
8   { nombre: "Juan Rodríguez", edad: 23, carrera: "Historia", semestre:  
9     6, promedio: 7.8 },  
10 ])
```

```
6 { nombre: "Elena Pérez", edad: 19, carrera: "Matemáticas", semestre:
  2, promedio: 9.5 }
7 ])
```

1.2.5.1 Ejemplos de consultas

- Obtener todos los estudiantes:

```
1 db.estudiantes.find({})
```

- Filtrar por carrera:

```
1 db.estudiantes.find({ carrera: "Biología" })
```

- Estudiantes mayores de 21 años:

```
1 db.estudiantes.find({ edad: { $gt: 21 } })
```

- Promedio mayor o igual a 8:

```
1 db.estudiantes.find({ promedio: { $gte: 8.0 } })
```

- Ordenar por promedio descendente:

```
1 db.estudiantes.find().sort({ promedio: -1 })
```

- Limitar resultados:

```
1 db.estudiantes.find().limit(3)
```

MongoDB ofrece una amplia variedad de operadores y opciones para realizar consultas avanzadas. Se recomienda consultar la documentación oficial para profundizar en estos aspectos.

1.3 Introducción: REST + MongoDB + SPA ligera

En este capítulo se aborda el desarrollo de una aplicación web completa basada en una **arquitectura cliente-servidor**, utilizando tecnologías ampliamente empleadas en el desarrollo de aplicaciones modernas. El objetivo es integrar un **backend REST** desarrollado con Spring Boot y MongoDB con un **frontend ligero** construido con HTML5, Bootstrap y JavaScript, siguiendo el modelo de **Single Page Application (SPA) simple**, sin necesidad de herramientas de construcción complejas.

1.3.1 Objetivo general del capítulo

El objetivo principal es que el alumnado sea capaz de **diseñar, implementar, probar y documentar** una aplicación que:

- Exponga una **API REST** para la gestión de datos.
- Utilice una **base de datos orientada a documentos (MongoDB)** para almacenar la información.
- Consuma dicha API desde un cliente web mediante **peticiones HTTP asíncronas**.
- Aplique buenas prácticas básicas de diseño, validación y manejo de errores.

Este enfoque permite evaluar de forma directa y práctica los criterios de evaluación relacionados con el acceso a datos y el desarrollo de aplicaciones multicapa.

1.3.2 Arquitectura de la aplicación

La aplicación se estructura en tres capas claramente diferenciadas:

- **Cliente (frontend)** Implementado con HTML5, Bootstrap y JavaScript (jQuery). Se encarga de la interacción con el usuario y del consumo de la API REST mediante peticiones AJAX.
- **Servidor (backend)** Desarrollado con Spring Boot, expone una API REST que gestiona la lógica de negocio y el acceso a la base de datos. Se apoya en Spring Data MongoDB para la persistencia de la información.
- **Base de datos** MongoDB almacena la información en forma de documentos BSON, permitiendo un modelo flexible que se adapta bien a los requisitos del dominio de reservas.

La comunicación entre cliente y servidor se realiza exclusivamente mediante **JSON sobre HTTP**, lo que desacopla ambas capas y facilita su mantenimiento y evolución.

1.3.3 Por qué REST, MongoDB y una SPA ligera

La combinación de estas tecnologías no es casual y responde a criterios técnicos y pedagógicos:

- **REST** permite definir una interfaz clara, estándar y fácilmente comprobable para el acceso a los datos, favoreciendo la separación de responsabilidades.
- **MongoDB**, como base de datos orientada a documentos, facilita el trabajo con estructuras de datos complejas y jerárquicas, alineándose bien con el modelo de reservas planteado.
- Una **SPA ligera**, sin frameworks pesados ni procesos de compilación, permite centrarse en los conceptos fundamentales del consumo de APIs y la gestión del estado en el cliente, reduciendo la carga cognitiva inicial.

Este enfoque resulta especialmente adecuado para comprender las diferencias entre bases de datos relacionales y documentales, así como para introducir patrones de desarrollo habituales en aplicaciones web actuales.

1.3.4 Caso práctico: gestión de reservas de pistas deportivas

A lo largo del capítulo se desarrollará una aplicación que permite:

- Gestionar **instalaciones deportivas**.
- Definir **horarios** asociados a dichas instalaciones, almacenando un “snapshot” de la información relevante.
- Registrar **usuarios**.
- Crear, consultar, modificar y cancelar **reservas**, combinando información embebida (horario e instalación) con referencias a otros documentos (usuario).

Para entender mejor la aplicación veamos el siguiente caso de uso:

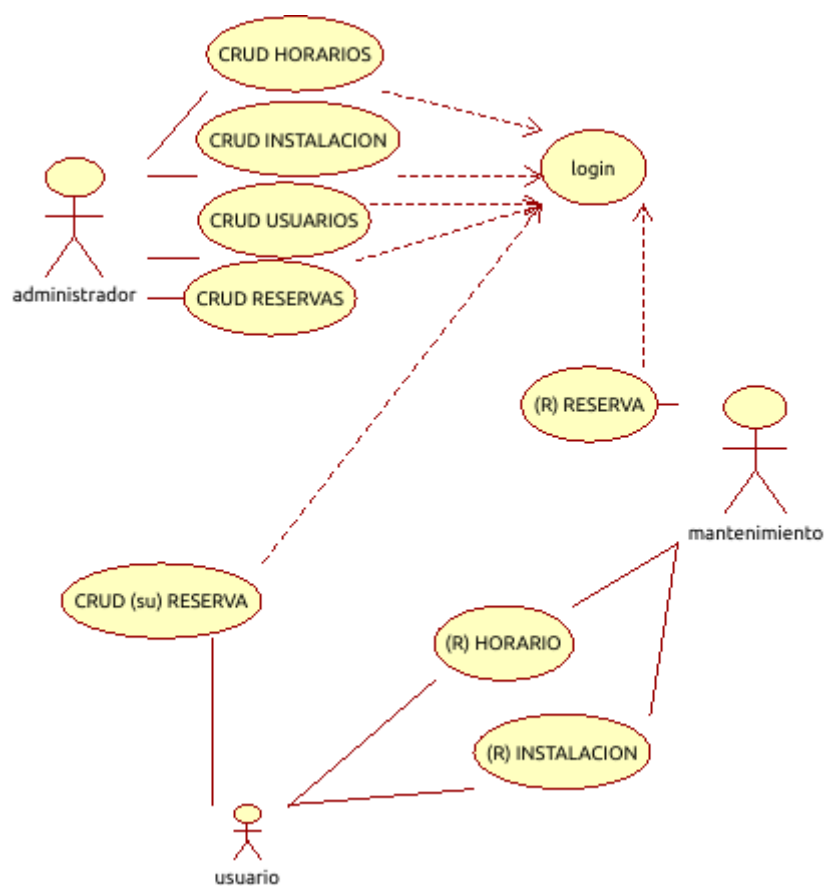
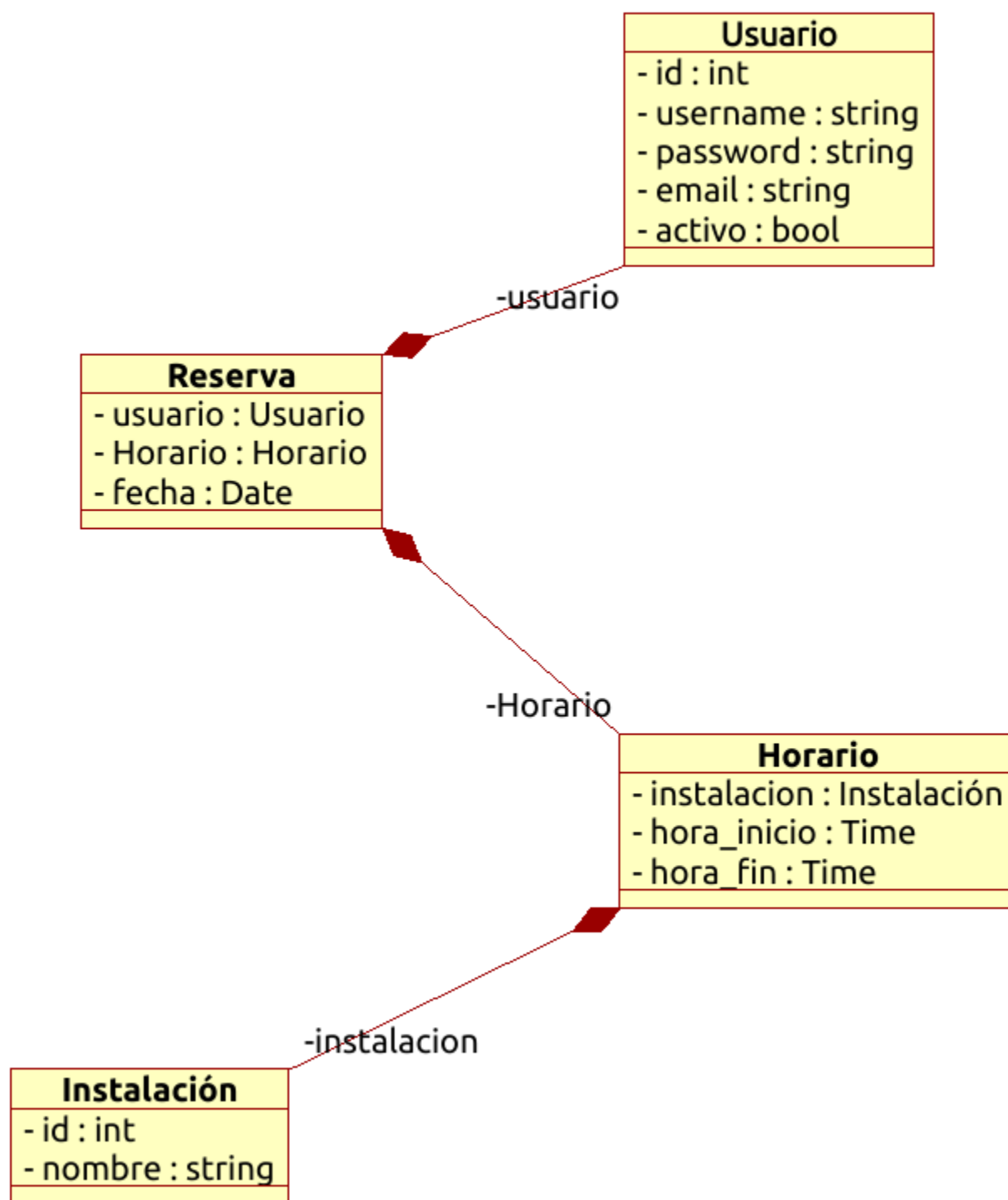


Figura 4: Diagrama de caso de uso de gestión de pistas deportivas

El diagrama UML que seguiremos es este:

**Figura 5:** Diagrama UML de Gestión de Pistas Deportivas

Este modelo ha sido elegido porque:

- Presenta **objetos simples y estructurados**, ideales para trabajar la persistencia en MongoDB.
- Requiere **consultas filtradas** (por fecha, usuario o instalación).
- Obliga a gestionar la **consistencia de los datos**, especialmente al evitar solapamientos de reservas, lo que introduce el concepto de transacción o control lógico de integridad.

1.4 Preparación del entorno y dependencias

En este apartado se prepara un entorno reproducible basado en **Dev Containers** para minimizar el tiempo de arranque y evitar problemas de configuración entre equipos. El objetivo es disponer, desde el primer momento, de:

- Un contenedor de desarrollo con **Java + Maven** para el backend.
- Un servicio **MongoDB** como base de datos documental.
- Un servicio **Mongo Express** (interfaz web) para inspección y administración.
- Un archivo **.env en la raíz del proyecto** con variables compartidas tanto por Docker Compose como por Spring Boot.
- Un proyecto **Spring Boot (Maven)** con los *starters* necesarios y configuración en **YAML**.

1.4.1 Requisitos previos

- Docker y Docker Compose instalados en el equipo.
- Visual Studio Code con la extensión **Dev Containers**.
- Git (recomendado) para versionar el proyecto.

1.4.2 Estructura de carpetas recomendada

Se propone la siguiente estructura mínima:

```
1 reservas-pistas/  
2   .devcontainer/  
3     devcontainer.json  
4     docker-compose.yml  
5   src/  
6   pom.xml  
7   .env  
8   application.yml (en src/main/resources)
```

La carpeta **.devcontainer/** contiene toda la configuración necesaria para levantar el entorno completo (IDE + servicios) con un único comando desde VS Code.

1.4.3 Creación del archivo .env (variables compartidas)

En la raíz del proyecto se crea un archivo **.env**. Este archivo centraliza configuración sensible y de entorno (usuario, password, nombre de base de datos, puertos).

Ejemplo recomendado:


```
1 # MongoDB (servicio Docker)
2 MONGO_PORT=27017
3 MONGO_INITDB_ROOT_USERNAME=root
4 MONGO_INITDB_ROOT_PASSWORD=83uddjfp0cmMD
5 MONGO_DB=reservas_db
6
7 # Mongo Express
8 MONGO_EXPRESS_PORT=8081
9 ME_CONFIG_BASICAUTH_USERNAME=mongo
10 ME_CONFIG_BASICAUTH_PASSWORD=83uddjfp0cmMD
```

Buenas prácticas:

- En un entorno real, este archivo no debería subirse al repositorio (usar `.gitignore`).
- En contexto educativo, puede incluirse si se pretende homogeneidad, pero conviene explicitarlo.

1.4.4 Preparación del Dev Container

La idea es que VS Code se conecte a un contenedor de desarrollo (“workspace”) y, a la vez, levante los servicios auxiliares (MongoDB y Mongo Express) mediante Docker Compose.

1.4.4.1 `.devcontainer/docker-compose.yml` Se define el contenedor de desarrollo y los servicios. Este archivo usa variables del `.env`.

```
1 version: "3.8"
2
3 services:
4   app:
5     image: mcr.microsoft.com/devcontainers/java:21
6     volumes:
7       - ../workspace:cached
8     working_dir: /workspace
9     command: sleep infinity
10    depends_on:
11      - mongo
12      - mongo-express
13    environment:
14      # Variables disponibles también dentro del contenedor de
15      # desarrollo
16      MONGO_INITDB_ROOT_USERNAME: ${MONGO_INITDB_ROOT_USERNAME}
17      MONGO_INITDB_ROOT_PASSWORD: ${MONGO_INITDB_ROOT_PASSWORD}
18      MONGO_DB: ${MONGO_DB}
19
20   mongo:
21     image: mongo:7
```

```
21 restart: "no"
22 environment:
23     MONGO_INITDB_ROOT_USERNAME: ${MONGO_INITDB_ROOT_USERNAME}
24     MONGO_INITDB_ROOT_PASSWORD: ${MONGO_INITDB_ROOT_PASSWORD}
25 ports:
26     - "${MONGO_PORT}:27017"
27 volumes:
28     - mongo_data:/data/db
29
30 mongo-express:
31     image: mongo-express:1
32     restart: "no"
33     depends_on:
34         - mongo
35     ports:
36         - "${MONGO_EXPRESS_PORT}:8081"
37     environment:
38         ME_CONFIG_BASICAUTH_USERNAME: ${ME_CONFIG_BASICAUTH_USERNAME}
39         ME_CONFIG_BASICAUTH_PASSWORD: ${ME_CONFIG_BASICAUTH_PASSWORD}
40         ME_CONFIG_MONGODB_ADMINUSERNAME: ${MONGO_INITDB_ROOT_USERNAME}
41         ME_CONFIG_MONGODB_ADMINPASSWORD: ${MONGO_INITDB_ROOT_PASSWORD}
42         ME_CONFIG_MONGODB_URL: mongodb://${MONGO_INITDB_ROOT_USERNAME}:${MONGO_INITDB_ROOT_PASSWORD}@mongo:27017/
43
44 volumes:
45     mongo_data:
```

Notas técnicas:

- Se fija MongoDB 7 por estabilidad y compatibilidad.
- Se añade volumen `mongo_data` para persistencia entre reinicios.
- El contenedor `app` no ejecuta la aplicación; simplemente proporciona el entorno de desarrollo.

1.4.4.2 .devcontainer/devcontainer.json Configura cómo VS Code se conecta al contenedor.

```
1 {
2     "name": "reservas-pistas-devcontainer",
3     "dockerComposeFile": "docker-compose.yml",
4     "service": "app",
5     "workspaceFolder": "/workspace",
6     "shutdownAction": "stopCompose",
7     "customizations": {
8         "vscode": {
9             "extensions": [
10                 "vscjava.vscode-java-pack",
11                 "vmware.vscode-spring-boot",
12                 "redhat.vscode-yaml"
13             ]
14         }
15     }
16 }
```

```
14     }  
15   }  
16 }
```

Con esto, al abrir el proyecto en VS Code:

1. Se elige “Reopen in Container”.
2. Se construye el entorno y se levantan los servicios.
3. MongoDB queda disponible desde el contenedor `app` por hostname `mongo:27017`.

1.4.5 Creación del proyecto Spring Boot (Maven)

El backend se implementará como un proyecto Maven estándar con Spring Boot.

1.4.5.1 Opción recomendada: Spring Initializr

1. Acceder a Spring Initializr (web) o usar integración en IDE.
2. Configurar:
 - Project: **Maven**
 - Language: **Java**
 - Spring Boot: versión estable
 - Group: `com.dam`
 - Artifact: `reservas`
 - Packaging: `jar`
 - Java: 21 (o 17 si el centro lo requiere)
3. Añadir dependencias (*starters*):
 - **Spring Web**
 - **Spring Data MongoDB**
 - **Validation**

Generar, descargar y descomprimir dentro del repositorio `reservas-pistas/`.

1.4.5.2 Alternativa: crear Maven y añadir starters manualmente

Si se parte de un Maven “vacío”:

1. Crear proyecto Maven:

```
1 mvn archetype:generate \  
2   -DgroupId=com.dam \  
3   -DartifactId=reservas \  
4   -DarchetypeArtifactId=maven-archetype-quickstart \  
5   -DinteractiveMode=false
```

2. Sustituir el `pom.xml` por uno basado en Spring Boot, incluyendo:

- Parent `spring-boot-starter-parent`
- Dependencias de `web`, `data-mongodb`, `validation`

En entornos docentes suele ser mejor usar Spring Initializr para evitar ruido.

1.4.6 Dependencias (starters) necesarias

En el `pom.xml` deben estar al menos:

- `spring-boot-starter-web`: API REST (controladores, JSON, HTTP).
- `spring-boot-starter-data-mongodb`: persistencia MongoDB.
- `spring-boot-starter-validation`: validación de DTOs con `@Valid`.

Ejemplo mínimo:

```
1 <dependencies>  
2   <dependency>  
3     <groupId>org.springframework.boot</groupId>  
4     <artifactId>spring-boot-starter-web</artifactId>  
5   </dependency>  
6  
7   <dependency>  
8     <groupId>org.springframework.boot</groupId>  
9     <artifactId>spring-boot-starter-data-mongodb</artifactId>  
10  </dependency>  
11  
12  <dependency>  
13    <groupId>org.springframework.boot</groupId>  
14    <artifactId>spring-boot-starter-validation</artifactId>  
15  </dependency>  
16 </dependencies>
```

1.4.7 Configuración en YAML (`application.yml`) usando variables de entorno

En Spring Boot se usará `application.yml` en `src/main/resources/`.

La idea es **leer las variables del `.env`**. En Docker Compose, el `.env` se carga automáticamente para los servicios. Para Spring, hay dos estrategias:

- Ejecutar Spring desde el contenedor `app` y pasarle variables de entorno (recomendado).
- Cargar `.env` con una librería adicional (opcional, no necesario si se usa Compose).

1.4.7.1 `src/main/resources/application.yml`

```
1 server:
2   port: 8080
3
4 spring:
5   data:
6     mongodb:
7       uri: mongodb://${MONGO_INITDB_ROOT_USERNAME}:${MONGO_INITDB_ROOT_PASSWORD}@mongo:27017/${MONGO_DB}?authSource=admin
8
9 logging:
10  level:
11    org.springframework.web: INFO
```

Notas:

- `mongo` es el hostname del servicio dentro de Docker Compose.
- `authSource=admin` es necesario cuando se autentica con usuario root.
- La base de datos que usará la aplicación será `${MONGO_DB}`.

1.4.8 Verificación del entorno

Una vez abierto el proyecto en Dev Container:

1. Comprobar que Mongo está levantado:

```
1 docker ps
```

2. Acceder a Mongo Express en el host:

- `http://localhost:8081`
- Usuario/contraseña definidos en `.env`

3. Probar conexión desde el contenedor de desarrollo:

```
1 mongosh "mongodb://root:83uddjfp0cmMD@mongo:27017/?authSource=admin"
```

1.4.9 Nota sobre “mongoengine”

MongoEngine es un ODM típico de Python (Flask/Django), no forma parte del stack Java/Spring. En Spring Boot, el equivalente conceptual es:

- **Spring Data MongoDB** (repositorios, mapeo de documentos, consultas derivadas)
- Alternativamente [MongoTemplate](#) para consultas más personalizadas

En este capítulo se utilizará **Spring Data MongoDB** por su alineación con el enfoque REST y por simplificar el CRUD y las consultas.

1.4.10 Archivo .env de ejemplo

El archivo `.env`, situado en la raíz del proyecto, centraliza las variables de entorno utilizadas tanto por Docker Compose como por Spring Boot. De este modo se evita la duplicación de configuración y se facilita la modificación del entorno sin tocar el código.

Ejemplo completo de `.env`:

```
1 # =====
2 # MongoDB
3 # =====
4 MONGO_PORT=27017
5 MONGO_INITDB_ROOT_USERNAME=root
6 MONGO_INITDB_ROOT_PASSWORD=83uddjfp0cmMD
7 MONGO_DB=reservas_db
8
9 # =====
10 # Mongo Express
11 # =====
12 MONGO_EXPRESS_PORT=8081
13 ME_CONFIG_BASICAUTH_USERNAME=mongo
14 ME_CONFIG_BASICAUTH_PASSWORD=83uddjfp0cmMD
```

Notas importantes:

- En un entorno profesional, este archivo **no debe versionarse**.
- En el contexto educativo puede compartirse para facilitar el arranque, pero debe explicarse el motivo y las implicaciones.
- Todas las variables definidas aquí están disponibles para los contenedores definidos en Docker Compose.

1.4.11 Archivo `.gitignore` recomendado

Para evitar subir al repositorio archivos generados automáticamente o que contengan información sensible, se recomienda incluir al menos las siguientes entradas en el archivo `.gitignore`:

```
1 # Variables de entorno
2 .env
3
4 # Directorios generados por Maven
5 target/
6
7 # Archivos de configuración del IDE
8 .idea/
9 *.iml
10 .vscode/
11
12 # Logs
13 *.log
```

Este `.gitignore` evita:

- La publicación accidental de credenciales.
- Conflictos entre distintos entornos de desarrollo.
- La inclusión de binarios o artefactos generados en tiempo de compilación.

1.4.12 Arranque de la aplicación Spring Boot desde el contenedor

Una vez que el proyecto se ha abierto en el **Dev Container**, el arranque de la aplicación se realiza desde el propio contenedor de desarrollo.

Desde el terminal integrado de VS Code:

```
1 ./mvnw spring-boot:run
```

Este comando:

- Utiliza el *Maven Wrapper* incluido en el proyecto.
- Arranca Spring Boot usando la configuración definida en `application.yml`.
- Conecta automáticamente con MongoDB usando las variables de entorno proporcionadas por Docker Compose.

Si todo es correcto, en la salida del log se podrá observar:

- Inicialización del contexto de Spring.
- Conexión correcta a MongoDB.
- Puerto HTTP del servidor web en estado “listening”.

1.4.13 Cheatsheet para troubleshooting

Ante problemas de arranque o conexión, conviene revisar de forma sistemática los siguientes puntos:

1.4.13.1 MongoDB no arranca o no responde

- Verificar que los contenedores están en ejecución:

```
1 docker ps
```

- Comprobar que el puerto definido en `.env` (`MONGO_PORT`) no está siendo usado por otro servicio.
- Revisar los logs del contenedor:

```
1 docker logs <id_contenedor_mongo>
```

1.4.13.2 Error de autenticación en MongoDB

- Confirmar que `MONGO_INITDB_ROOT_USERNAME` y `MONGO_INITDB_ROOT_PASSWORD` coinciden en:

- `.env`
- `docker-compose.yml`
- `application.yml`

- Asegurarse de que la URI de conexión incluye:

```
1 authSource=admin
```

1.4.13.3 Spring Boot no conecta con MongoDB

- Verificar que en `application.yml` se usa el hostname `mongo` y no `localhost`.
- Comprobar que Spring se está ejecutando **dentro del contenedor**, no en el host.
- Probar conexión manual desde el contenedor:

```
1 mongosh "mongodb://root:password@mongo:27017/?authSource=admin"
```


1.4.13.4 Mongo Express no es accesible

- Confirmar que el puerto `MONGO_EXPRESS_PORT` está correctamente expuesto.
- Acceder desde el navegador a:

```
1 http://localhost:8081
```

- Revisar credenciales de acceso web (basic auth).

1.4.13.5 Cambios en `.env` no surten efecto

- Recordar que Docker Compose **no recarga automáticamente** variables de entorno.
- Detener y volver a levantar los servicios:

```
1 docker-compose down
2 docker-compose up -d
```

1.5 MongoDB: documentos, colecciones y ObjectId

MongoDB es una base de datos **orientada a documentos**, lo que implica un modelo de almacenamiento y acceso a los datos diferente al de las bases de datos relacionales tradicionales. En este apartado se introducen los conceptos fundamentales necesarios para comprender cómo se almacenan y gestionan los datos en MongoDB y cómo estos conceptos se aplican al sistema de **reservas de pistas deportivas** que se desarrollará a lo largo del capítulo.

1.5.1 Documentos

El **documento** es la unidad básica de almacenamiento en MongoDB. Un documento es una estructura de datos en formato **BSON** (Binary JSON), similar a un objeto JSON, que permite:

- Pares clave–valor.
- Tipos de datos simples (string, number, boolean, date).
- Estructuras anidadas (objetos dentro de objetos).
- Arrays de valores u objetos.

A diferencia de una fila en una tabla relacional, un documento puede contener información jerárquica y no requiere un esquema rígido previamente definido.

Ejemplo de documento sencillo:

```
1 {  
2   "_id": ObjectId("65a1c9a1f1c2e0a9a0b12345"),  
3   "nombre": "Pista Central",  
4   "direccion": "C/ Mayor 1",  
5   "ciudad": "Jaén"  
6 }
```

En este ejemplo, todos los datos relacionados con una instalación deportiva se encuentran agrupados en un único documento.

1.5.2 Colecciones

Una **colección** es un conjunto de documentos del mismo tipo lógico. Conceptualmente, puede compararse con una tabla en una base de datos relacional, aunque con diferencias importantes:

- Los documentos de una colección **no están obligados a tener la misma estructura exacta**.
- No es necesario crear la colección previamente; MongoDB la crea automáticamente al insertar el primer documento.
- No existen claves foráneas ni restricciones de integridad a nivel de base de datos.

Ejemplos de colecciones en el sistema de reservas:

- `instalaciones`
- `usuarios`
- `reservas`

Cada una de estas colecciones almacena documentos relacionados con una entidad del dominio de la aplicación.

1.5.3 ObjectId

Cada documento en MongoDB tiene un identificador único denominado `_id`. Si no se especifica, MongoDB genera automáticamente un **ObjectId**.

Un **ObjectId** es un valor de 12 bytes que incluye:

- Marca temporal de creación.
- Identificador del proceso.
- Contador incremental.

Esto garantiza unicidad sin necesidad de coordinación central.

Ejemplo de `_id`:

```
1 "_id": ObjectId("65a1c9a1f1c2e0a9a0b12345")
```

En aplicaciones Java con Spring Data MongoDB, el **ObjectId** suele mapearse a un campo de tipo `String` o **ObjectId**, siendo habitual el uso de `String` para simplificar la serialización y el trabajo con JSON.

1.5.4 Documentos embebidos y referencias

Uno de los aspectos más importantes del diseño en MongoDB es decidir **qué datos se embeben** dentro de un documento y **qué datos se referencian** mediante identificadores.

1.5.4.1 Documentos embebidos Un documento embebido es un objeto que se almacena dentro de otro documento. Este enfoque es adecuado cuando:

- Los datos se consultan siempre junto al documento principal.
- No se necesita reutilizar el subdocumento de forma independiente.
- Se desea preservar un “snapshot” del estado de los datos en un momento concreto.

En el sistema de reservas, el **horario** se embebe dentro de la reserva, y a su vez contiene una copia de la instalación.

Ejemplo simplificado de documento `reserva` con documentos embebidos:

```
1 {
2   "_id": ObjectId("65a2aa0f9d3e9c1b8c111111"),
3   "fechaReserva": "2026-01-18T10:15:00Z",
4   "usuarioId": "65a199f3e0b1c2d3e4f56789",
5   "horario": {
6     "dia": "2026-01-20",
7     "horaInicio": "18:00",
8     "horaFin": "19:00",
9     "instalacionSnapshot": {
10      "instalacionId": "65a18888aa11223344556677",
11      "nombre": "Pista Central",
12      "direccion": "C/ Mayor 1",
13      "ciudad": "Jaén"
14    }
15  }
16 }
```

Este diseño permite que una reserva conserve la información de la instalación tal y como era en el momento de la reserva, incluso si la instalación cambia posteriormente.

1.5.4.2 Referencias entre documentos Las **referencias** consisten en almacenar el identificador (`_id`) de otro documento en lugar de embebido completo. Este enfoque se utiliza cuando:

- El documento referenciado se reutiliza en muchos contextos.
- La información debe mantenerse siempre actualizada.
- No es necesario cargarla automáticamente en cada consulta.

En este proyecto, la reserva **referencia al usuario** mediante su identificador (`usuarioId`), evitando duplicar datos personales.

Ejemplo de referencia:

```
1 {
2   "usuarioId": "65a199f3e0b1c2d3e4f56789"
3 }
```

La recuperación de los datos completos del usuario se realiza, si es necesario, mediante una consulta adicional desde la aplicación.

1.5.5 Comparación con el modelo relacional

Aspecto	Relacional	MongoDB (Documental)
Unidad básica	Fila	Documento
Agrupación	Tabla	Colección
Identificador	Clave primaria	<code>_id</code> (ObjectId)
Relaciones	JOIN	Embebido o referencia
Esquema	Rígido	Flexible

Esta comparación ayuda a comprender por qué MongoDB resulta especialmente adecuado para modelos de dominio complejos y jerárquicos, como el sistema de reservas planteado.

1.6 Spring Data MongoDB: @Document, repositorios y consultas

En este apartado se prepara la capa de persistencia de la aplicación de **reservas de pistas deportivas** utilizando **Spring Data MongoDB**. El objetivo es dejar definidos:

- Los **documentos** (clases de dominio) mapeados a MongoDB.
- Los **repositorios** (`MongoRepository`) para operaciones CRUD.
- Un conjunto inicial de **consultas** típicas que se utilizarán más adelante en la API REST.

En el siguiente apartado se construirán los **DTOs** y los **endpoints**, por lo que aquí se evita exponer directamente las entidades como contrato de la API.

1.6.1 Dependencias necesarias

Este apartado asume que el proyecto ya incluye el *starter*:

- `spring-boot-starter-data-mongodb`

y que la conexión está configurada en `application.yml` con la URI construida mediante variables de entorno.

1.6.2 Diseño del modelo y decisiones de persistencia

Partimos del diagrama proporcionado, con estas decisiones explícitas:

- `Instalacion` es un **documento independiente** (colección `instalaciones`).
- `Usuario` es un **documento independiente** (colección `usuarios`).
- `Reserva` es un **documento independiente** (colección `reservas`).
- `Reserva` **embebe** `Horario`.
- `Horario` **embebe** un “snapshot” de la instalación (en vez de referenciarla directamente).
- `Reserva` **referencia** a `Usuario` mediante `usuarioId` (String que representa el `_id`).

Esto permite:

- Consultar reservas sin necesidad de joins.
- Mantener en la reserva los datos de instalación tal como estaban en el momento de la reserva.
- Evitar duplicar datos de usuario en cada reserva.

1.6.3 Paquetes recomendados

Para mantener orden y escalabilidad:

```
1 com.dam.reservas
2     model
3     repo
```

1.6.4 1) Documentos y embebidos

1.6.4.1 model/Instalacion.java

```
1 package com.dam.reservas.model;
2
3 import jakarta.validation.constraints.NotBlank;
4 import org.springframework.data.annotation.Id;
5 import org.springframework.data.mongodb.core.mapping.Document;
6
7 @Document(collection = "instalaciones")
8 public class Instalacion {
9
10     @Id
11     private String id;
12
13     @NotBlank(message = "nombre es obligatorio")
14     private String nombre;
15
16     @NotBlank(message = "direccion es obligatoria")
17     private String direccion;
18
19     @NotBlank(message = "ciudad es obligatoria")
20     private String ciudad;
21
22     public Instalacion() {}
23
24     public Instalacion(String nombre, String direccion, String ciudad) {
25         this.nombre = nombre;
26         this.direccion = direccion;
27         this.ciudad = ciudad;
28     }
29
30     public String getId() { return id; }
31     public void setId(String id) { this.id = id; }
32
33     public String getNombre() { return nombre; }
34     public void setNombre(String nombre) { this.nombre = nombre; }
35
36     public String getDireccion() { return direccion; }
37     public void setDireccion(String direccion) { this.direccion =
        direccion; }
38
39     public String getCiudad() { return ciudad; }
40     public void setCiudad(String ciudad) { this.ciudad = ciudad; }
41 }
```

1.6.4.2 model/Usuario.java

```
1 package com.dam.reservas.model;
2
3 import jakarta.validation.constraints.Email;
4 import jakarta.validation.constraints.NotBlank;
5 import org.springframework.data.annotation.Id;
6 import org.springframework.data.mongodb.core.mapping.Document;
7
8 @Document(collection = "usuarios")
9 public class Usuario {
10
11     @Id
12     private String id;
13
14     @NotBlank(message = "nombre es obligatorio")
15     private String nombre;
16
17     @NotBlank(message = "email es obligatorio")
18     @Email(message = "email no válido")
19     private String email;
20
21     public Usuario() {}
22
23     public Usuario(String nombre, String email) {
24         this.nombre = nombre;
25         this.email = email;
26     }
27
28     public String getId() { return id; }
29     public void setId(String id) { this.id = id; }
30
31     public String getNombre() { return nombre; }
32     public void setNombre(String nombre) { this.nombre = nombre; }
33
34     public String getEmail() { return email; }
35     public void setEmail(String email) { this.email = email; }
36 }
```

1.6.4.3 model/InstalacionSnapshot.java (embebido) Este subdocumento es la “foto” de la instalación incluida dentro de la reserva, cumpliendo la composición del diagrama.

```
1 package com.dam.reservas.model;
2
3 import jakarta.validation.constraints.NotBlank;
4
5 public class InstalacionSnapshot {
6
7     @NotBlank(message = "instalacionId es obligatorio")
8     private String instalacionId;
```



```
9
10 @NotBlank(message = "nombre es obligatorio")
11 private String nombre;
12
13 @NotBlank(message = "direccion es obligatoria")
14 private String direccion;
15
16 @NotBlank(message = "ciudad es obligatoria")
17 private String ciudad;
18
19 public InstalacionSnapshot() {}
20
21 public InstalacionSnapshot(String instalacionId, String nombre,
22     String direccion, String ciudad) {
23     this.instalacionId = instalacionId;
24     this.nombre = nombre;
25     this.direccion = direccion;
26     this.ciudad = ciudad;
27 }
28
29 public String getInstalacionId() { return instalacionId; }
30 public void setInstalacionId(String instalacionId) { this.
31     instalacionId = instalacionId; }
32
33 public String getNombre() { return nombre; }
34 public void setNombre(String nombre) { this.nombre = nombre; }
35
36 public String getDireccion() { return direccion; }
37 public void setDireccion(String direccion) { this.direccion =
38     direccion; }
39
40 public String getCiudad() { return ciudad; }
41 public void setCiudad(String ciudad) { this.ciudad = ciudad; }
42 }
```

1.6.4.4 model/Horario.java (embebido) Se define como objeto embebido, no como documento. Para facilitar consultas y validación, se modela con tipos Java adecuados:

- `LocalDate` para el día.
- `LocalTime` para horas.

```
1 package com.dam.reservas.model;
2
3 import jakarta.validation.Valid;
4 import jakarta.validation.constraints.NotNull;
5
6 import java.time.LocalDate;
7 import java.time.LocalTime;
8
```

```
9 public class Horario {
10
11     @NotNull(message = "dia es obligatorio")
12     private LocalDate dia;
13
14     @NotNull(message = "horaInicio es obligatoria")
15     private LocalTime horaInicio;
16
17     @NotNull(message = "horaFin es obligatoria")
18     private LocalTime horaFin;
19
20     @NotNull(message = "instalacionSnapshot es obligatorio")
21     @Valid
22     private InstalacionSnapshot instalacionSnapshot;
23
24     public Horario() {}
25
26     public Horario(LocalDate dia, LocalTime horaInicio, LocalTime horaFin
27         , InstalacionSnapshot instalacionSnapshot) {
28         this.dia = dia;
29         this.horaInicio = horaInicio;
30         this.horaFin = horaFin;
31         this.instalacionSnapshot = instalacionSnapshot;
32     }
33
34     public LocalDate getDia() { return dia; }
35     public void setDia(LocalDate dia) { this.dia = dia; }
36
37     public LocalTime getHoraInicio() { return horaInicio; }
38     public void setHoraInicio(LocalTime horaInicio) { this.horaInicio =
39         horaInicio; }
40
41     public LocalTime getHoraFin() { return horaFin; }
42     public void setHoraFin(LocalTime horaFin) { this.horaFin = horaFin; }
43
44     public InstalacionSnapshot getInstalacionSnapshot() { return
45         instalacionSnapshot; }
46     public void setInstalacionSnapshot(InstalacionSnapshot
47         instalacionSnapshot) { this.instalacionSnapshot =
48         instalacionSnapshot; }
49
50     public boolean rangoValido() {
51         return horaInicio != null && horaFin != null && horaInicio.isBefore
52             (horaFin);
53     }
54 }
```

1.6.4.5 model/Reserva.java Se define *Reserva* como documento. Incluye:

- *fechaReserva*: marca temporal del momento en que se registró la reserva.

- `horario`: embebido.
- `usuarioId`: referencia al usuario.

```
1 package com.dam.reservas.model;  
2  
3 import jakarta.validation.Valid;  
4 import jakarta.validation.constraints.NotBlank;  
5 import jakarta.validation.constraints.NotNull;  
6 import org.springframework.data.annotation.Id;  
7 import org.springframework.data.mongodb.core.mapping.Document;  
8  
9 import java.time.Instant;  
10  
11 @Document(collection = "reservas")  
12 public class Reserva {  
13  
14     @Id  
15     private String id;  
16  
17     @NotNull(message = "fechaReserva es obligatoria")  
18     private Instant fechaReserva;  
19  
20     @NotNull(message = "horario es obligatorio")  
21     @Valid  
22     private Horario horario;  
23  
24     @NotBlank(message = "usuarioId es obligatorio")  
25     private String usuarioId;  
26  
27     public Reserva() {}  
28  
29     public Reserva(Instant fechaReserva, Horario horario, String  
30         usuarioId) {  
31         this.fechaReserva = fechaReserva;  
32         this.horario = horario;  
33         this.usuarioId = usuarioId;  
34     }  
35  
36     public String getId() { return id; }  
37     public void setId(String id) { this.id = id; }  
38  
39     public Instant getFechaReserva() { return fechaReserva; }  
40     public void setFechaReserva(Instant fechaReserva) { this.fechaReserva  
41         = fechaReserva; }  
42  
43     public Horario getHorario() { return horario; }  
44     public void setHorario(Horario horario) { this.horario = horario; }  
45  
46     public String getUsuarioId() { return usuarioId; }  
47     public void setUsuarioId(String usuarioId) { this.usuarioId =  
48         usuarioId; }
```

```
46 }
```

1.6.5 2) Repositorios (MongoRepository) y consultas

Los repositorios proporcionan CRUD automático y permiten definir consultas por convención de nombres, sin necesidad de escribir SQL.

1.6.5.1 repo/InstalacionRepository.java Consultas típicas:

- Por ciudad
- Búsqueda parcial por nombre

```
1 package com.dam.reservas.repo;
2
3 import com.dam.reservas.model.Instalacion;
4 import org.springframework.data.mongodb.repository.MongoRepository;
5
6 import java.util.List;
7
8 public interface InstalacionRepository extends MongoRepository<
9     Instalacion, String> {
10
11     List<Instalacion> findByCiudadIgnoreCase(String ciudad);
12
13     List<Instalacion> findByNombreContainingIgnoreCase(String q);
14 }
```

1.6.5.2 repo/UsuarioRepository.java Consultas típicas:

- Buscar por email (único lógico)

```
1 package com.dam.reservas.repo;
2
3 import com.dam.reservas.model.Usuario;
4 import org.springframework.data.mongodb.repository.MongoRepository;
5
6 import java.util.Optional;
7
8 public interface UsuarioRepository extends MongoRepository<Usuario,
9     String> {
10
11     Optional<Usuario> findByEmailIgnoreCase(String email);
12 }
```

1.6.5.3 repo/ReservaRepository.java Consultas típicas del dominio:

- Reservas por usuario.
- Reservas por instalación (usando el snapshot embebido).
- Reservas por día.
- Reservas por instalación y día (base para comprobar solapes).

```
1 package com.dam.reservas.repo;
2
3 import com.dam.reservas.model.Reserva;
4 import org.springframework.data.mongodb.repository.MongoRepository;
5
6 import java.time.LocalDate;
7 import java.util.List;
8
9 public interface ReservaRepository extends MongoRepository<Reserva,
10     String> {
11     List<Reserva> findByUsuarioId(String usuarioId);
12
13     List<Reserva> findByHorario_InstalacionSnapshot_InstalacionId(String
14         instalacionId);
15
16     List<Reserva> findByHorario_Dia(LocalDate dia);
17
18     List<Reserva>
19         findByHorario_InstalacionSnapshot_InstalacionIdAndHorario_Dia(
20             String instalacionId, LocalDate dia);
21 }
```

1.6.6 Consideraciones de consulta y modelado

1.6.6.1 Consultar por datos embebidos MongoDB permite consultar campos dentro de documentos anidados. Spring Data lo refleja mediante el uso del guion bajo en el nombre del método:

- `findByHorario_Dia(...)`
- `findByHorario_InstalacionSnapshot_InstalacionId(...)`

Esto evita escribir consultas manuales y favorece la claridad del código.

1.6.6.2 Recomendación sobre índices (preparación para rendimiento) Aunque no es imprescindible en un entorno docente inicial, en un proyecto real se crean índices para acelerar consultas frecuentes. Dado que se consultará con mucha frecuencia por:

- `usuarioId`

- `horario.dia`
- `horario.instalacionSnapshot.instalacionId`

En apartados posteriores se puede introducir `@Indexed` en el documento `Reserva` si se desea.

1.6.7 Estado del proyecto tras este apartado

Al finalizar este apartado, el proyecto queda preparado con:

- Clases de dominio mapeadas a MongoDB con `@Document`.
- Objetos embebidos definidos conforme al diagrama.
- Repositorios con CRUD y consultas base.

En el siguiente apartado comenzaremos:

- **DTOs de entrada y salida**
- **endpoints REST**
- validación a nivel de API y códigos HTTP

De este modo, se evita que las entidades de persistencia se utilicen directamente como contrato de la API, manteniendo una separación clara entre capas.

1.7 Diseño de la API REST: endpoints, DTOs, códigos HTTP

En este apartado se define el **contrato REST** de la aplicación de reservas de pistas deportivas: qué recursos existen, qué endpoints se exponen, qué datos se intercambian mediante **DTOs**, y qué **códigos HTTP** se deben devolver en cada operación. Esta definición es clave para desacoplar el frontend del backend y para garantizar una API coherente, predecible y fácilmente testeable.

El diseño se basa en el modelo ya establecido:

- **Instalacion** (documento independiente)
- **Usuario** (documento independiente)
- **Reserva** (documento principal), que embebe **Horario** y el snapshot de instalación, y referencia a usuario mediante **usuarioId**

1.7.1 Principios de diseño aplicados

1. **Recursos como sustantivos** Las rutas representan recursos (**/reservas**, **/usuarios**, **/instalaciones**) y las operaciones se expresan con métodos HTTP.
2. **JSON como formato de intercambio** Todas las peticiones y respuestas utilizan JSON.
3. **Separación entre entidades y DTOs** Las clases del paquete **model** no se exponen directamente en la API. Se diseñan DTOs específicos de entrada y salida.
4. **Códigos HTTP coherentes** Cada endpoint debe responder con el código más apropiado, evitando respuestas ambiguas.
5. **Validación en el borde (API)** Los DTOs de entrada se validan con **@Valid** y anotaciones de Bean Validation para evitar guardar datos inválidos.

1.7.2 Recursos y endpoints

1.7.2.1 1) Recurso: Instalaciones Las instalaciones representan las pistas o instalaciones deportivas disponibles.

1.7.2.1.1 Endpoints propuestos

Método	Ruta	Descripción	Respuesta
GET	<code>/api/instalaciones</code>	Listar instalaciones (con filtros opcionales)	200 + lista
GET	<code>/api/instalaciones/{id}</code>	Obtener una instalación	200 / 404
POST	<code>/api/instalaciones</code>	Crear instalación	201 / 400
PUT	<code>/api/instalaciones/{id}</code>	Actualizar instalación	200 / 400 / 404
DELETE	<code>/api/instalaciones/{id}</code>	Eliminar instalación	204 / 404

Filtros recomendados:

- `?ciudad=Jaén`
- `?q=pista` (búsqueda por nombre)

1.7.2.2 2) Recurso: Usuarios Los usuarios se almacenan como documentos independientes y se referencian desde la reserva por `usuarioId`.

1.7.2.2.1 Endpoints propuestos

Método	Ruta	Descripción	Respuesta
GET	<code>/api/usuarios</code>	Listar usuarios	200 + lista
GET	<code>/api/usuarios/{id}</code>	Obtener usuario	200 / 404
POST	<code>/api/usuarios</code>	Crear usuario	201 / 400 / 409
PUT	<code>/api/usuarios/{id}</code>	Actualizar usuario	200 / 400 / 404 / 409
DELETE	<code>/api/usuarios/{id}</code>	Eliminar usuario	204 / 404

Se recomienda tratar `email` como identificador lógico único (aunque el índice único se puede introducir más adelante como extensión).

1.7.2.3 3) Recurso: Reservas La reserva es el recurso principal del sistema.

- Embebe `Horario`
- `Horario` embebe `InstalacionSnapshot`
- Referencia a usuario por `usuarioId`

1.7.2.3.1 Endpoints propuestos

Método	Ruta	Descripción	Respuesta
GET	<code>/api/reservas</code>	Listar reservas (con filtros)	200 + lista
GET	<code>/api/reservas/{id}</code>	Obtener una reserva	200 / 404
POST	<code>/api/reservas</code>	Crear reserva	201 / 400 / 404 / 409
PUT	<code>/api/reservas/{id}</code>	Actualizar reserva (reemplazo)	200 / 400 / 404 / 409
DELETE	<code>/api/reservas/{id}</code>	Eliminar reserva	204 / 404

Filtros recomendados para `GET /api/reservas`:

- `?usuarioId=...`
- `?instalacionId=...`
- `?dia=2026-01-18`
- combinación: `?instalacionId=...&dia=...` (base para evitar solapes)

1.7.2.3.2 Conflicto por solape (409) Si se intenta crear/modificar una reserva que se solapa con otra reserva existente de la misma instalación y mismo día, el endpoint debe responder:

- **409 Conflict**
- cuerpo JSON con detalle del motivo

1.7.3 DTOs (Data Transfer Objects)

Se definen DTOs de entrada (request) y salida (response). Los DTOs de entrada se diseñan con los campos mínimos necesarios, evitando que el cliente tenga que construir manualmente objetos embebidos como el snapshot de instalación. El backend se encarga de construir el snapshot consultando la instalación.

1.7.3.1 DTOs para Instalaciones

1.7.3.1.1 InstalacionRequest Campos requeridos para crear/actualizar:

- `nombre`
- `direccion`
- `ciudad`

1.7.3.1.2 InstalacionResponse Incluye además el `id`.

1.7.3.2 DTOs para Usuarios

1.7.3.2.1 UsuarioRequest

- `nombre`
- `email`

1.7.3.2.2 UsuarioResponse Incluye `id`.

1.7.3.3 DTOs para Reservas La reserva necesita:

- Referencia a usuario (`usuarioId`)
- Referencia a instalación (`instalacionId`) para construir el snapshot en el backend
- Día y franja horaria

1.7.3.3.1 ReservaCreateRequest (entrada)

- `usuarioId`
- `instalacionId`
- `dia`

- `horaInicio`
- `horaFin`

1.7.3.3.2 ReservaResponse (salida)

- `id`
- `fechaReserva` (timestamp)
- `usuarioId`
- `horario` con snapshot de instalación embebido

Se recomienda que el response incluya el horario embebido completo, ya que es precisamente el valor añadido del modelo documental.

1.7.4 Códigos HTTP y comportamiento esperado

1.7.4.1 Respuestas típicas

- **200 OK** Operación correcta en GET/PUT.
- **201 Created** Recurso creado correctamente (POST). Recomendación: devolver el recurso creado.
- **204 No Content** Eliminación correcta (DELETE) sin cuerpo de respuesta.
- **400 Bad Request** Datos inválidos (validación), formato incorrecto o parámetros incoherentes.
- **404 Not Found** Recurso inexistente (por ejemplo `usuarioId` inválido al reservar).
- **409 Conflict** Violación de regla de negocio, típicamente:
 - Solape de reserva
 - Email duplicado (si se impone unicidad)

1.7.4.2 Estructura de errores Para mantener uniformidad, se recomienda una estructura JSON única para los errores, gestionada por `@ControllerAdvice`:

```
1 {
2   "timestamp": "2026-01-18T18:45:00Z",
3   "status": 400,
4   "error": "VALIDATION",
5   "message": "Datos no válidos",
6   "details": [
7     "día: es obligatorio",
8     "horaFin: debe ser posterior a horaInicio"
9   ]
10 }
```

1.7.5 Ejemplos de intercambio (contrato)

1.7.5.1 Crear reserva (POST /api/reservas) Request:

```
1 {  
2   "usuarioId": "65a199f3e0b1c2d3e4f56789",  
3   "instalacionId": "65a18888aa11223344556677",  
4   "dia": "2026-01-18",  
5   "horaInicio": "18:00",  
6   "horaFin": "19:00"  
7 }
```

Respuestas:

- 201 si se crea
- 404 si el usuario o la instalación no existen
- 409 si la franja se solapa

1.7.6 Preparación para el siguiente apartado

Tras definir el contrato REST, el siguiente paso es implementarlo:

1. Crear los DTOs de entrada/salida.
2. Mapear entidades ↔ DTOs.
3. Implementar controladores y servicios aplicando:
 - Validación (`@Valid`)
 - Códigos HTTP correctos
 - Gestión uniforme de errores (`ControllerAdvice`)
 - Regla de negocio de solape (409 Conflict)

En el siguiente apartado se desarrollarán los **DTOs concretos y su validación**, dejando los controladores listos para implementar los endpoints de la API.

1.8 Validación, DTOs y Control de Errores (@ControllerAdvice)

En el desarrollo de una API REST profesional, no basta con que el código “funcione”. Debe ser robusto, seguro y fácil de mantener. Para ello, nos apoyamos en dos pilares fundamentales: el patrón **DTO** para la transferencia de datos y un **Manejador Global de Excepciones**.

En una API REST, la validación de datos y la gestión homogénea de errores son elementos esenciales para garantizar que el sistema:

- Rechaza entradas inválidas antes de persistirlas.
- Responde con **códigos HTTP correctos** y mensajes consistentes.
- Facilita el consumo desde el frontend (manejo de errores predecible).
- Simplifica el mantenimiento: los controladores no se llenan de **try/catch**.

1.8.1 1. Entendiendo el patrón DTO (Data Transfer Object)

Antes de implementar el código, es crucial entender qué es un DTO y por qué no debemos usar nuestras Entidades (@Entity) directamente en los controladores.

¿Qué es un DTO? Un DTO es un objeto simple (POJO o Java Record) que transporta datos de un proceso a otro. No tiene lógica de negocio ni acceso a base de datos; es puramente un contenedor de datos.

¿Por qué lo usamos en lugar de la Entidad?

1. **Desacoplamiento (Seguridad):** Tu base de datos no debe exponerse directamente al cliente. Una entidad puede tener campos sensibles (como `password`, `fechaCreacion`, `flagBorrado`) que el cliente no debe ver ni manipular. El DTO actúa como un filtro.
2. **Validación Específica:** Las reglas de validación de la API (ej. “el email debe tener formato válido”) suelen ser diferentes a las de la base de datos (ej. “el email no puede ser nulo”). Usar DTOs permite poner anotaciones `@NotBlank` o `@Email` sin ensuciar la entidad JPA.
3. **Independencia del Modelo:** Si mañana cambias la estructura de tu base de datos (divides una tabla en dos), no rompes la API que consumen los clientes, porque el DTO se mantiene estable y tú te encargas de transformar los datos internamente.
4. **Optimización:** Puedes crear diferentes DTOs para la misma entidad. Por ejemplo, `UsuarioResumenDTO` (solo nombre y foto) para un listado, y `UsuarioDetalleDTO` (todos los datos) para la vista de perfil.

El Flujo de Trabajo:

1. **Request:** El cliente envía un JSON Spring lo convierte a **DTO**.
2. **Mapping:** El controlador o servicio convierte ese **DTO** a una **Entidad**.

3. **Persistencia:** La **Entidad** se guarda en la BBDD.

1.8.2 2. Implementación de Validación en DTOs

La validación se realiza en los **DTOs de entrada** (request). Esto permite rechazar peticiones mal formadas antes de que lleguen siquiera a procesarse.

Este diagrama de secuencia muestra cómo un dato “crudo” (JSON) se transforma en un DTO validado y, finalmente, en una Entidad persistente. Ilustra la separación de responsabilidades:

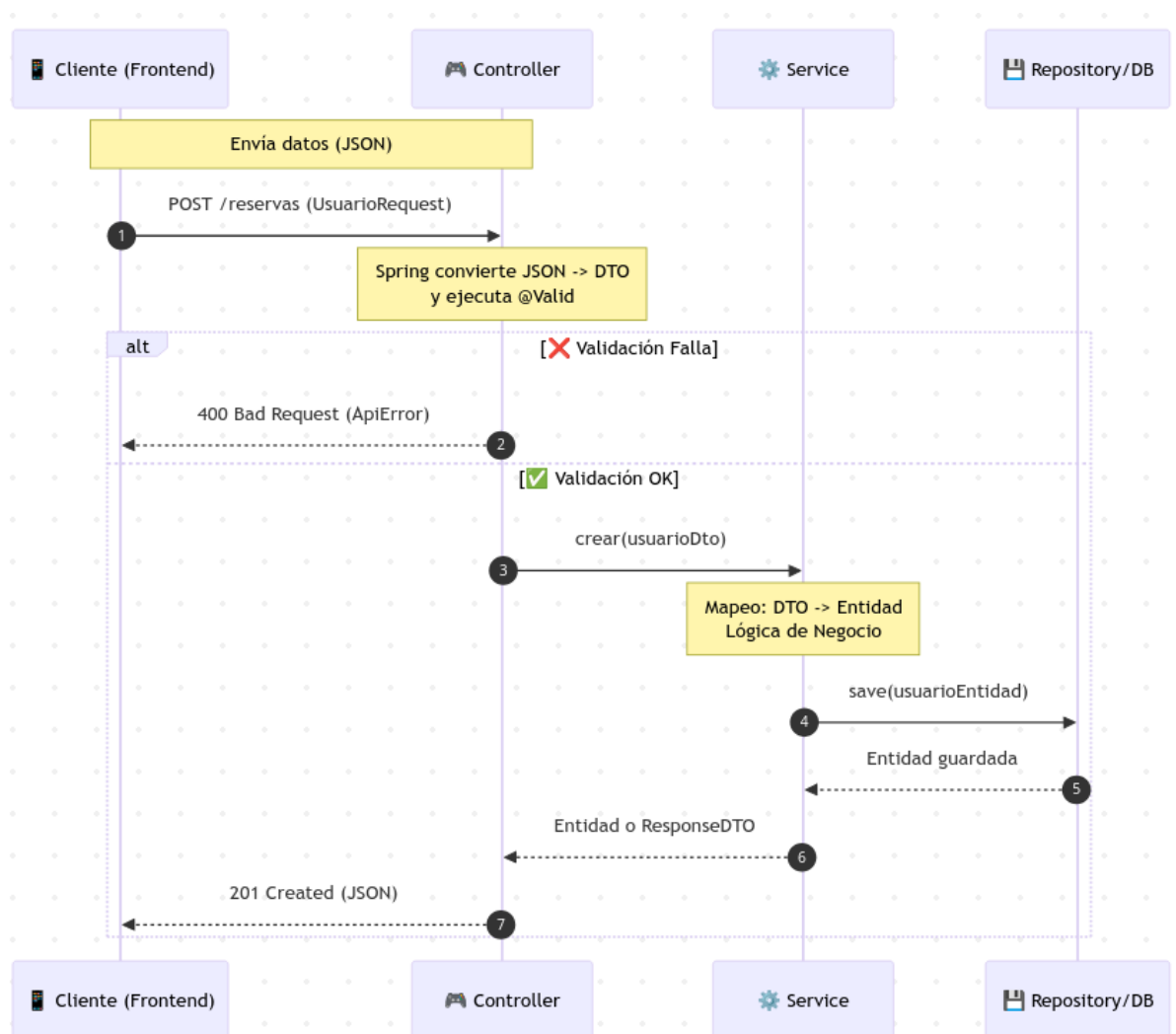


Figura 6: Diagrama secuencia conversión JSON-Entidad

1 sequenceDiagram

```

2    autonumber
3    participant Cliente as Cliente (Frontend)
4    participant Ctrl as Controller
5    participant Srv as Service
6    participant Repo as Repository/DB
7
8    Note over Cliente, Ctrl: Envía datos (JSON)
9    Cliente->>Ctrl: POST /reservas (UsuarioRequest)
10
11    Note over Ctrl: Spring convierte JSON -> DTO<br/>y ejecuta @Valid
12
13    alt Validación Falla
14        Ctrl-->>Cliente: 400 Bad Request (ApiError)
15    else Validación OK
16        Ctrl->>Srv: crear(usuarioDto)
17
18        Note over Srv: Mapeo: DTO -> Entidad<br/>Lógica de Negocio
19
20        Srv->>Repo: save(usuarioEntidad)
21        Repo-->>Srv: Entidad guardada
22
23        Srv-->>Ctrl: Entidad o ResponseDTO
24        Ctrl-->>Cliente: 201 Created (JSON)
25    end

```

1.8.2.1 Dependencia necesaria Asegúrate de tener en `pom.xml`:

- `spring-boot-starter-validation`

1.8.2.2 Activación en el Controlador Usamos dos anotaciones clave:

- `@RequestBody`: Le dice a Spring “toma el JSON del cuerpo de la petición y conviértelo a este objeto Java”.
- `@Valid`: Le dice a Spring “antes de entrar al método, revisa que este objeto cumpla todas las restricciones definidas en su clase”.

```

1 @PostMapping
2 @ResponseStatus(HttpStatus.CREATED)
3 // Si 'req' no cumple las validaciones, el método NO se ejecuta
4 // y salta una excepción automática.
5 public ReservaResponse crear(@Valid @RequestBody ReservaCreateRequest
    req) { ... }

```

1.8.2.3 Definición de DTOs con `jakarta.validation` A continuación, los DTOs usando Java Records (inmutables y concisos) y anotaciones de validación.

dto/InstalacionRequest.java

```
1 package com.dam.reservas.dto;
2
3 import jakarta.validation.constraints.NotBlank;
4
5 public record InstalacionRequest(
6     @NotBlank(message = "nombre es obligatorio") String nombre,
7     @NotBlank(message = "direccion es obligatoria") String direccion,
8     @NotBlank(message = "ciudad es obligatoria") String ciudad
9 ) {}
```

dto/UsuarioRequest.java

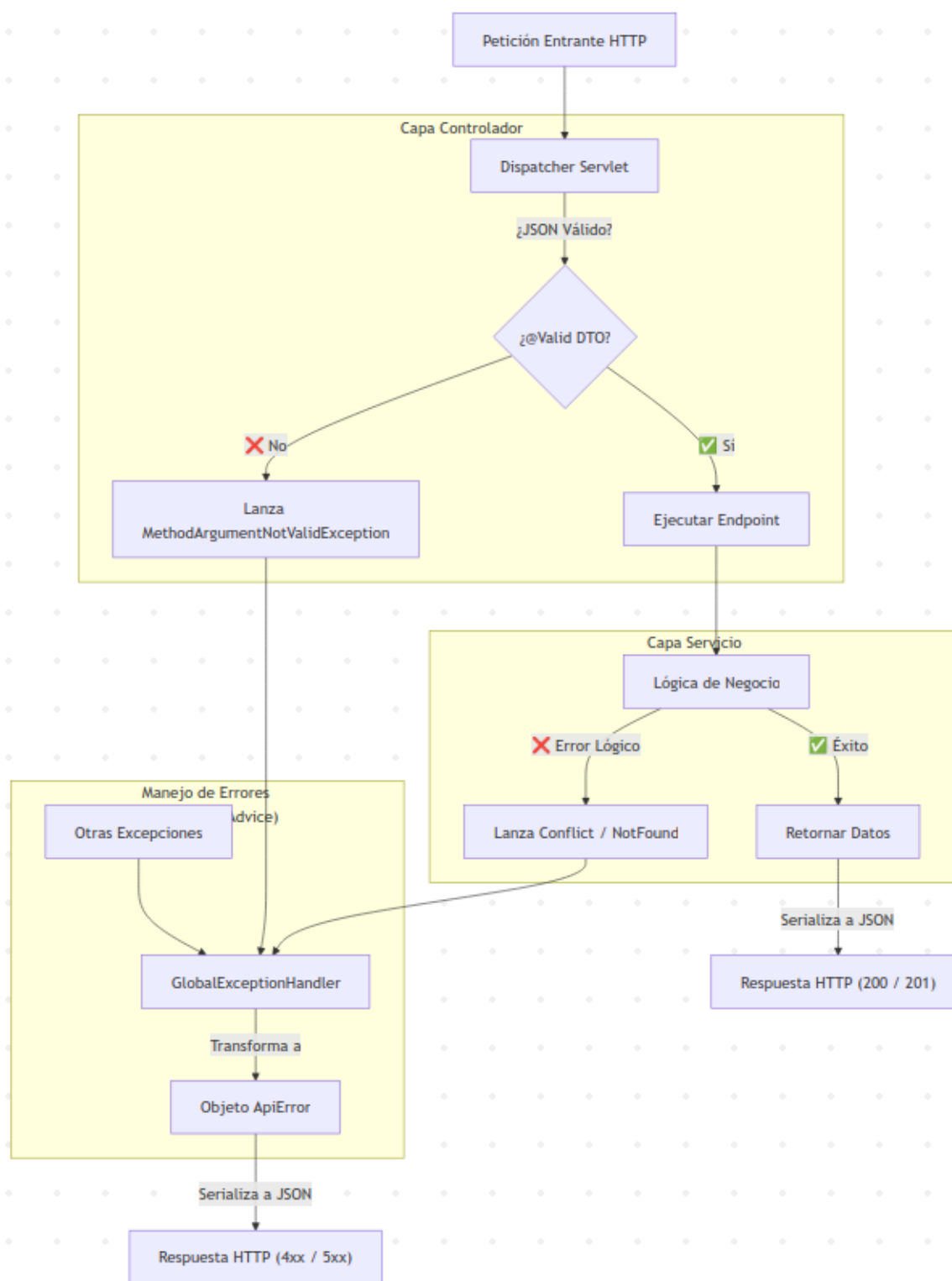
```
1 package com.dam.reservas.dto;
2
3 import jakarta.validation.constraints.Email;
4 import jakarta.validation.constraints.NotBlank;
5
6 public record UsuarioRequest(
7     @NotBlank(message = "nombre es obligatorio") String nombre,
8     @NotBlank(message = "email es obligatorio") @Email(message = "email
9         no válido") String email
10 ) {}
```

dto/ReservaCreateRequest.java *Nota:* Aquí validamos formato y presencia. Las reglas de negocio complejas (ej. “hora fin mayor que hora inicio”) se validan en el Servicio.

```
1 package com.dam.reservas.dto;
2
3 import jakarta.validation.constraints.NotBlank;
4 import jakarta.validation.constraints.NotNull;
5 import java.time.LocalDate;
6 import java.time.LocalDateTime;
7
8 public record ReservaCreateRequest(
9     @NotBlank(message = "usuarioId es obligatorio") String usuarioId,
10    @NotBlank(message = "instalacionId es obligatorio") String
11        instalacionId,
12    @NotNull(message = "dia es obligatorio") LocalDate dia,
13    @NotNull(message = "horaInicio es obligatoria") LocalDateTime
14        horaInicio,
15    @NotNull(message = "horaFin es obligatoria") LocalDateTime horaFin
16 ) {}
```


1.8.3 3. Manejo Centralizado de Excepciones

En una aplicación tradicional, tendríamos bloques **try-catch** repetidos en cada controlador, lo cual ensucia el código y hace difícil mantener una respuesta de error coherente.

**Figura 7:** Diagrama de flujo para el manejo de excepciones

```

1 flowchart TD
2     req["Petición Entrante HTTP"] --> dispatcher["Dispatcher Servlet"]
3
4     subgraph Capa_Controlador ["Capa Controlador"]
5         dispatcher -->|?`JSON Válido?| valid{"?`@Valid DTO?"}
6         valid -->| No| exValid["Lanza MethodArgumentNotValidException"]
7         valid -->| Sí| ctrl["Ejecutar Endpoint"]
8     end
9
10    subgraph Capa_Servicio ["Capa Servicio"]
11        ctrl --> service["Lógica de Negocio"]
12        service -->| Error Lógico| exBiz["Lanza Conflict / NotFound"]
13        service -->| Éxito| success["Retornar Datos"]
14    end
15
16    subgraph Manejo_Errores ["Manejo de Errores (@ControllerAdvice)"]
17        exValid --> handler["GlobalExceptionHandler"]
18        exBiz --> handler
19        exGenerica["Otras Excepciones"] --> handler
20
21        handler -->|Transforma a| apiError["Objeto ApiError"]
22    end
23
24    apiError -->|Serializa a JSON| response["Respuesta HTTP (4xx / 5xx)"]
25    success -->|Serializa a JSON| response2["Respuesta HTTP (200 / 201)"]

```

Spring Boot ofrece una solución elegante: **@ControllerAdvice**.

¿Cómo funciona? Funciona mediante Programación Orientada a Aspectos (AOP). Es un componente que “escucha” a todos los controladores. Si un controlador lanza una excepción (ej. `NotFoundException`), el flujo normal se interrumpe y el `@ControllerAdvice` captura esa excepción, permitiéndote generar una respuesta JSON personalizada.

Beneficios:

- **Código Limpio:** Los controladores solo se ocupan del “camino feliz” (cuando todo va bien).
- **Consistencia:** Todos los errores (404, 400, 500) tienen exactamente el mismo formato JSON, facilitando la vida al desarrollador del Frontend.

1.8.3.1 3.1. Modelo de error unificado (ApiError) Definimos una “plantilla” de cómo se verán todos nuestros errores.

```

1 package com.dam.reservas.web;
2
3 import java.time.Instant;

```

```
4 import java.util.List;
5
6 public record ApiError(
7     Instant timestamp, // Cuándo ocurrió
8     int status,         // Código HTTP (400, 404, etc.)
9     String error,       // Nombre corto del error
10    String message,      // Mensaje legible para humanos
11    List<String> details // Detalles técnicos o lista de campos
                        // fallidos
12 ) {}
```

1.8.3.2 3.2. Excepciones de Dominio Creamos nuestras propias excepciones semánticas que heredan de `RuntimeException`. Esto nos permite lanzar errores que “significan algo” en nuestro negocio.

web/NotFoundException.java (Para recursos no encontrados - 404)

```
1 package com.dam.reservas.web;
2 public class NotFoundException extends RuntimeException {
3     public NotFoundException(String message) { super(message); }
4 }
```

web/BadRequestException.java (Para peticiones mal formadas - 400)

```
1 package com.dam.reservas.web;
2 public class BadRequestException extends RuntimeException {
3     public BadRequestException(String message) { super(message); }
4 }
```

web/ConflictException.java (Para reglas de negocio rotas, ej. solapamiento - 409)

```
1 package com.dam.reservas.web;
2 public class ConflictException extends RuntimeException {
3     public ConflictException(String message) { super(message); }
4 }
```

1.8.3.3 3.3. Implementación del GlobalExceptionHandler Aquí es donde ocurre la magia. Mapeamos cada excepción Java a una respuesta HTTP concreta.

web/GlobalExceptionHandler.java

```
1 package com.dam.reservas.web;
2
3 import org.springframework.http.HttpStatus;
4 import org.springframework.http.ResponseEntity;
5 import org.springframework.validation.FieldError;
6 import org.springframework.web.bind.MethodArgumentNotValidException;
```

```
7 import org.springframework.web.bind.annotation.*;
8
9 import java.time.Instant;
10 import java.util.List;
11
12 @ControllerAdvice
13 public class GlobalExceptionHandler {
14
15     // 1. Manejo de Recurso No Encontrado (404)
16     @ExceptionHandler(NotFoundException.class)
17     public ResponseEntity<ApiError> handleNotFound(NotFoundException ex)
18     {
19         return ResponseEntity.status(HttpStatus.NOT_FOUND).body(
20             new ApiError(Instant.now(), 404, "NOT_FOUND", ex.getMessage(),
21                 List.of())
22         );
23     }
24
25     // 2. Manejo de Reglas de Negocio Generales (400)
26     @ExceptionHandler(BadRequestException.class)
27     public ResponseEntity<ApiError> handleBadRequest(BadRequestException
28     ex) {
29         return ResponseEntity.status(HttpStatus.BAD_REQUEST).body(
30             new ApiError(Instant.now(), 400, "BAD_REQUEST", ex.getMessage()
31                 , List.of())
32         );
33     }
34
35     // 3. Manejo de Conflictos (409) - Ej: Reserva duplicada
36     @ExceptionHandler(ConflictException.class)
37     public ResponseEntity<ApiError> handleConflict(ConflictException ex)
38     {
39         return ResponseEntity.status(HttpStatus.CONFLICT).body(
40             new ApiError(Instant.now(), 409, "CONFLICT", ex.getMessage(),
41                 List.of())
42         );
43     }
44
45     // 4. Manejo de Validación de DTOs (@Valid falla)
46     // Spring lanza MethodArgumentNotValidException cuando @Valid detecta
47     // errores.
48     @ExceptionHandler(MethodArgumentNotValidException.class)
49     public ResponseEntity<ApiError> handleValidation(
50         MethodArgumentNotValidException ex) {
51
52         // Extraemos los errores campo por campo para informar al frontend
53         List<String> details = ex.getBindingResult().getAllErrors().stream
54             ()
55             .map(err -> {
56                 if (err instanceof FieldError fe) {
57                     return fe.getField() + ": " + fe.getDefaultMessage();
58                 }
59             })
60             .collect(Collectors.toList());
61
62         return ResponseEntity.status(HttpStatus.BAD_REQUEST).body(
63             new ApiError(Instant.now(), 400, "VALIDATION_ERROR", details,
64                 List.of())
65         );
66     }
67 }
```

```

49         }
50         return err.getDefaultMessage();
51     })
52     .toList();
53
54     return ResponseEntity.status(HttpStatus.BAD_REQUEST).body(
55         new ApiError(Instant.now(), 400, "VALIDATION", "Datos de
           entrada inválidos", details)
56     );
57 }
58
59 // 5. Manejo de Errores Inesperados (500)
60 // Captura cualquier otra excepción no controlada (NullPointerException, SQL
   error, etc.)
61 @ExceptionHandler(Exception.class)
62 public ResponseEntity<ApiError> handleGeneric(Exception ex) {
63     // IMPORTANTE: En producción, no mostrar ex.getMessage() si
       contiene info sensible.
64     return ResponseEntity.status(HttpStatus.INTERNAL_SERVER_ERROR).body(
65         (
66             new ApiError(Instant.now(), 500, "INTERNAL_ERROR", "Error
               interno del servidor", List.of(ex.getMessage()))
67         );
68 }

```

1.8.4 4. Resumen de Códigos de Estado HTTP

Al implementar estas validaciones y excepciones, tu API responderá semánticamente:

Código HTTP	Significado	Cuándo ocurre
201 Created	Creado	Reserva creada exitosamente.
400 Bad Request	Petición Incorrecta	Falla <code>@Valid</code> (campo vacío) o regla lógica simple (hora fin < hora inicio).
404 Not Found	No Encontrado	ID de usuario o instalación no existen en BBDD.
409 Conflict	Conflicto	Intento de reservar una pista que ya está ocupada.

Código HTTP	Significado	Cuándo ocurre
500 Internal Error	Error Servidor	Fallo de conexión a BBDD o error de programación no controlado.

1.8.5 Conclusiones

Al integrar este código, tu aplicación:

1. Rechaza automáticamente JSONs inválidos gracias a los **DTOs validados**.
2. Captura errores de lógica de negocio lanzados desde los Servicios.
3. Devuelve siempre una estructura JSON uniforme (**ApiError**) que el Frontend puede parsear fácilmente para mostrar alertas al usuario.

En el siguiente apartado se implementará **CORS y configuración por entornos**.

1.9 Validación y control de errores (@ControllerAdvice)

En una API REST, la validación de datos y la gestión homogénea de errores son elementos esenciales para garantizar que el sistema:

- Rechaza entradas inválidas antes de persistirlas.
- Responde con **códigos HTTP correctos** y mensajes consistentes.
- Facilita el consumo desde el frontend (manejo de errores predecible).
- Simplifica el mantenimiento: los controladores no se llenan de **try/catch**.

En este apartado se implementan dos bloques:

1. **Validación de DTOs** mediante Bean Validation (`jakarta.validation`) y `@Valid`.
2. **Manejo centralizado de excepciones** mediante `@ControllerAdvice`.

1.9.1 1) Validación de entrada en la API

La validación se realiza en los **DTOs de entrada** (request), no en las entidades del modelo persistente. Esto permite:

- Separar contrato de API y persistencia.
- Validar solo lo que el cliente debe enviar.
- Mantener reglas coherentes aunque la persistencia evolucione.

1.9.1.1 Dependencia necesaria Debe estar incluida la dependencia:

- `spring-boot-starter-validation`

1.9.1.2 Activación de validación en controladores En cada endpoint que reciba datos, se debe usar:

- `@Valid` en el parámetro del DTO.
- `@RequestBody` para deserializar JSON.

Ejemplo conceptual:

```
1 @PostMapping
2 @ResponseStatus(HttpStatus.CREATED)
3 public ReservaResponse crear(@Valid @RequestBody ReservaCreateRequest
    req) { ... }
```

Si el DTO no cumple las restricciones, Spring lanzará automáticamente una `MethodArgumentNotValidException`.

1.9.2 2) DTOs con anotaciones de Bean Validation

A continuación se incluyen los DTOs mínimos para el dominio, con validación.

1.9.2.1 dto/InstalacionRequest.java

```
1 package com.dam.reservas.dto;
2
3 import jakarta.validation.constraints.NotBlank;
4
5 public record InstalacionRequest(
6     @NotBlank(message = "nombre es obligatorio") String nombre,
7     @NotBlank(message = "direccion es obligatoria") String direccion,
8     @NotBlank(message = "ciudad es obligatoria") String ciudad
9 ) {}
```

1.9.2.2 dto/UsuarioRequest.java

```
1 package com.dam.reservas.dto;
2
3 import jakarta.validation.constraints.Email;
4 import jakarta.validation.constraints.NotBlank;
5
6 public record UsuarioRequest(
7     @NotBlank(message = "nombre es obligatorio") String nombre,
8     @NotBlank(message = "email es obligatorio") @Email(message = "email
9         no válido") String email
10 ) {}
```

1.9.2.3 dto/ReservaCreateRequest.java Se valida que los campos existan. La relación `horaInicio < horaFin` es una regla de negocio; se validará en la capa de servicio para poder responder con 400/409 según corresponda.

```
1 package com.dam.reservas.dto;
2
3 import jakarta.validation.constraints.NotBlank;
4 import jakarta.validation.constraints.NotNull;
5
6 import java.time.LocalDate;
7 import java.time.LocalTime;
8
9 public record ReservaCreateRequest(
10     @NotBlank(message = "usuarioId es obligatorio") String usuarioId,
11     @NotBlank(message = "instalacionId es obligatorio") String
12         instalacionId,
13     @NotNull(message = "dia es obligatorio") LocalDate dia,
14     @NotNull(message = "horaInicio es obligatoria") LocalTime
15         horaInicio,
16     @NotNull(message = "horaFin es obligatoria") LocalTime horaFin
17 ) {}
```

```
15 ) {}
```

1.9.3 3) Modelo de error unificado

Se define un formato de respuesta común para los errores. Esto permite al frontend:

- Mostrar mensajes coherentes.
- Leer detalles campo a campo cuando procede.
- Diferenciar tipos de error (`VALIDATION`, `NOT_FOUND`, etc.).

1.9.3.1 web/ApiError.java

```
1 package com.dam.reservas.web;  
2  
3 import java.time.Instant;  
4 import java.util.List;  
5  
6 public record ApiError(  
7     Instant timestamp,  
8     int status,  
9     String error,  
10    String message,  
11    List<String> details  
12 ) {}
```

1.9.4 4) Excepciones de dominio

Se definen excepciones específicas que se lanzarán desde la capa de servicio:

- `NotFoundException` → 404
- `BadRequestException` → 400
- `ConflictException` → 409 (por solapes u otras reglas)

1.9.4.1 web/NotFoundException.java

```
1 package com.dam.reservas.web;  
2  
3 public class NotFoundException extends RuntimeException {  
4     public NotFoundException(String message) { super(message); }  
5 }
```

1.9.4.2 web/BadRequestException.java

```
1 package com.dam.reservas.web;  
2  
3 public class BadRequestException extends RuntimeException {
```

```
4 public BadRequestException(String message) { super(message); }
5 }
```

1.9.4.3 web/ConflictException.java

```
1 package com.dam.reservas.web;
2
3 public class ConflictException extends RuntimeException {
4     public ConflictException(String message) { super(message); }
5 }
```

1.9.5 5) Manejo centralizado con @ControllerAdvice

Con `@ControllerAdvice` se interceptan excepciones lanzadas en cualquier controlador y se transforman en respuestas HTTP con el formato establecido.

1.9.5.1 web/GlobalExceptionHandler.java

```
1 package com.dam.reservas.web;
2
3 import org.springframework.http.HttpStatus;
4 import org.springframework.http.ResponseEntity;
5 import org.springframework.validation.FieldError;
6 import org.springframework.web.bind.MethodArgumentNotValidException;
7 import org.springframework.web.bind.annotation.*;
8
9 import java.time.Instant;
10 import java.util.List;
11
12 @ControllerAdvice
13 public class GlobalExceptionHandler {
14
15     @ExceptionHandler(NotFoundException.class)
16     public ResponseEntity<ApiError> handleNotFound(NotFoundException ex)
17     {
18         return ResponseEntity.status(HttpStatus.NOT_FOUND).body(
19             new ApiError(Instant.now(), 404, "NOT_FOUND", ex.getMessage(),
20                 List.of())
21         );
22     }
23
24     @ExceptionHandler(BadRequestException.class)
25     public ResponseEntity<ApiError> handleBadRequest(BadRequestException
26     ex) {
27         return ResponseEntity.status(HttpStatus.BAD_REQUEST).body(
28             new ApiError(Instant.now(), 400, "BAD_REQUEST", ex.getMessage(),
29                 List.of())
30         );
31     }
32 }
```

```
28
29 @ExceptionHandler(ConflictException.class)
30 public ResponseEntity<ApiError> handleConflict(ConflictException ex)
31 {
32     return ResponseEntity.status(HttpStatus.CONFLICT).body(
33         new ApiError(Instant.now(), 409, "CONFLICT", ex.getMessage(),
34             List.of())
35     );
36 }
37
38 @ExceptionHandler(MethodArgumentNotValidException.class)
39 public ResponseEntity<ApiError> handleValidation(
40     MethodArgumentNotValidException ex) {
41
42     List<String> details = ex.getBindingResult().getAllErrors().stream
43     ()
44     .map(err -> {
45         if (err instanceof FieldError fe) {
46             return fe.getField() + ": " + fe.getDefaultMessage();
47         }
48         return err.getDefaultMessage();
49     })
50     .toList();
51
52     return ResponseEntity.status(HttpStatus.BAD_REQUEST).body(
53         new ApiError(Instant.now(), 400, "VALIDATION", "Datos no vá
54         lidos", details)
55     );
56 }
57
58 @ExceptionHandler(Exception.class)
59 public ResponseEntity<ApiError> handleGeneric(Exception ex) {
60     // En un entorno docente puede resultar útil devolver el mensaje.
61     // En producción se recomienda no exponer detalles internos.
62     return ResponseEntity.status(HttpStatus.INTERNAL_SERVER_ERROR).body(
63         (
64             new ApiError(Instant.now(), 500, "INTERNAL_ERROR", "Error
65             interno del servidor", List.of(ex.getMessage()))
66         )
67     );
68 }
69 }
```

1.9.6 6) Reglas de negocio y códigos HTTP

No toda validación es “campo requerido”. En una aplicación real aparecen reglas como:

- `horaInicio` debe ser anterior a `horaFin` → **400 Bad Request**
- No se puede reservar una pista si ya hay una reserva solapada → **409 Conflict**

- No se puede reservar para un usuario inexistente → **404 Not Found**

Estas reglas se implementarán en la capa de servicio y lanzarán las excepciones definidas, para que el `@ControllerAdvice` las convierta en respuestas coherentes.

1.9.7 Resultado de este apartado

Al finalizar este apartado se dispone de:

- DTOs de entrada con validación.
- Un formato de error uniforme (`ApiError`).
- Excepciones de dominio.
- Un `@ControllerAdvice` que convierte errores en respuestas HTTP coherentes.

En el siguiente apartado se implementará **CORS y configuración por entornos**, garantizando que el frontend pueda consumir la API de forma segura y controlada en desarrollo y despliegue.

1.10 CORS y configuración por entornos

En una arquitectura cliente-servidor donde el frontend (HTML/JS) consume una API REST, es habitual que el navegador aplique la política de seguridad **Same-Origin Policy**. Esta política bloquea, por defecto, peticiones AJAX a un origen distinto (dominio, puerto o protocolo diferentes). Para permitir el acceso controlado a la API se utiliza **CORS (Cross-Origin Resource Sharing)**.

Además, en proyectos reales se requiere distinguir configuraciones según el entorno:

- **Desarrollo** (Dev Container, puertos locales, logs más verbosos).
- **Producción** (orígenes permitidos restringidos, credenciales seguras, logs moderados).

En este apartado se configura:

1. CORS para permitir el consumo desde el frontend.
2. Configuración por entornos con perfiles (**dev**, **prod**) manteniendo el uso de **.env**.

1.10.1 1) Conceptos básicos de CORS

CORS es un mecanismo basado en cabeceras HTTP que permite a un servidor indicar qué orígenes están autorizados a realizar solicitudes.

Elementos habituales:

- **Access-Control-Allow-Origin**: origen permitido.
- **Access-Control-Allow-Methods**: métodos permitidos (GET, POST, etc.).
- **Access-Control-Allow-Headers**: cabeceras permitidas.
- Petición **preflight** (OPTIONS): el navegador la envía antes de algunas solicitudes para comprobar permisos.

En desarrollo, si el frontend se abre, por ejemplo, desde:

- **http://localhost:5500** (Live Server) y la API está en:
- **http://localhost:8080**

son orígenes distintos por el puerto y el navegador bloqueará la llamada si no hay CORS configurado.

1.10.2 2) Estrategias posibles en este proyecto

En este capítulo se trabajará con dos escenarios comunes:

- **Escenario A (más simple):** el frontend se sirve desde Spring Boot (`src/main/resources/static`) y el origen es el mismo. En este caso, CORS no es estrictamente necesario, pero conviene conocerlo porque no siempre será así.
- **Escenario B (habitual en prácticas):** el frontend se sirve desde un servidor distinto (Live Server, Nginx, etc.). Aquí sí es necesario habilitar CORS.

Se implementará una configuración que permita **activar/desactivar** el origen permitido mediante variables de entorno.

1.10.3 3) Configuración de CORS en Spring Boot

Se define una clase de configuración implementando `WebMvcConfigurer` para aplicar CORS de forma global a `/api/**`.

1.10.3.1 config/CorsConfig.java

```
1 package com.dam.reservas.config;
2
3 import org.springframework.beans.factory.annotation.Value;
4 import org.springframework.context.annotation.Configuration;
5 import org.springframework.web.servlet.config.annotation.*;
6
7 @Configuration
8 public class CorsConfig implements WebMvcConfigurer {
9
10     @Value("${app.cors.allowed-origins:*}")
11     private String allowedOrigins;
12
13     @Override
14     public void addCorsMappings(CorsRegistry registry) {
15         registry.addMapping("/api/**")
16             .allowedOrigins(allowedOrigins.split(","))
17             .allowedMethods("GET", "POST", "PUT", "PATCH", "DELETE", "OPTIONS")
18             .allowedHeaders("*")
19             .allowCredentials(false)
20             .maxAge(3600);
21     }
22 }
```

Notas:

- `allowedOrigins` se puede definir como una lista separada por comas.
- En desarrollo puede ser `*` (no recomendado en producción).
- `allowCredentials(false)` evita combinaciones inseguras con `*`.

1.10.4 4) Configuración por entornos con perfiles

Spring Boot permite definir distintos ficheros YAML por perfil:

- `application.yml` (común)
- `application-dev.yml`
- `application-prod.yml`

La idea es mantener en `application.yml` lo esencial y delegar ajustes por perfil (CORS, logs, etc.) a los YAML específicos.

1.10.4.1 4.1 `application.yml` (base) Ubicación: `src/main/resources/application.yml`

```
1  server:
2    port: 8080
3
4  spring:
5    profiles:
6      active: ${SPRING_PROFILES_ACTIVE:dev}
7
8  data:
9    mongodb:
10     uri: mongodb://${MONGO_INITDB_ROOT_USERNAME}:${MONGO_INITDB_ROOT_PASSWORD}@mongo:27017/${MONGO_DB}?authSource=admin
11
12 app:
13   cors:
14     allowed-origins: ${APP_CORS_ALLOWED_ORIGINS:*}
```

- Se activa por defecto el perfil `dev` si no se define `SPRING_PROFILES_ACTIVE`.
- La URI de MongoDB se construye con variables del `.env` y el hostname `mongo` (servicio Compose).

1.10.4.2 4.2 `application-dev.yml` (desarrollo)

```
1  logging:
2    level:
3      org.springframework.web: INFO
4      org.springframework.data.mongodb: INFO
5
6  app:
7    cors:
8      allowed-origins: ${APP_CORS_ALLOWED_ORIGINS:*}
```

En desarrollo se suelen permitir orígenes más amplios y logs más verbosos.

1.10.4.3 4.3 application-prod.yml (producción)

```
1 logging:
2   level:
3     org.springframework.web: WARN
4     org.springframework.data.mongodb: WARN
5
6 app:
7   cors:
8     allowed-origins: ${APP_CORS_ALLOWED_ORIGINS:https://mi-dominio-
      ejemplo}
```

En producción:

- Se restringen orígenes permitidos a dominios concretos.
- Se moderan los logs.

1.10.5 5) Variables de entorno necesarias

En el `.env` se pueden definir variables adicionales para controlar CORS y perfil:

```
1 SPRING_PROFILES_ACTIVE=dev
2 APP_CORS_ALLOWED_ORIGINS=http://localhost:5500,http://localhost:8080
```

En desarrollo, si se usa Live Server, se añade su origen:

- `http://localhost:5500` (puede variar según configuración)

Si el frontend se sirve desde Spring (`/static`), bastaría con `http://localhost:8080`.

1.10.6 6) Verificación de CORS

Para comprobar CORS, se puede:

- Abrir la consola del navegador y verificar que no hay errores del tipo:
 - “blocked by CORS policy”
- Probar con `curl` simulando un origen:

```
1 curl -i http://localhost:8080/api/instalaciones \
2   -H "Origin: http://localhost:5500"
```

Debe observarse en la respuesta una cabecera `Access-Control-Allow-Origin` adecuada.

1.10.7 Resultado de este apartado

Tras este apartado el proyecto queda con:

- CORS configurado globalmente para `/api/**`.
- Control del origen permitido mediante variables de entorno (`APP_CORS_ALLOWED_ORIGINS`).
- Separación por entornos con perfiles (`dev / prod`) y YAML específico.
- Preparación para servir frontend desde un origen distinto sin problemas de navegador.

En el siguiente apartado se abordará: **“Front con Bootstrap: estructura y componentes”**, donde se definirá la estructura del cliente web y la interfaz para gestionar instalaciones, usuarios y reservas.

1.11 Front con Bootstrap: estructura y componentes

En este apartado se diseña el frontend de la aplicación como una interfaz web ligera basada en **HTML5 + Bootstrap 5**. El objetivo no es construir una SPA compleja, sino una interfaz clara y funcional que permita:

- Consultar instalaciones, usuarios y reservas.
- Crear y eliminar registros.
- Crear reservas indicando usuario, instalación y franja horaria.
- Mostrar mensajes de estado (éxito, error, validación).
- Preparar la estructura para que, en el siguiente apartado, se conecte la interfaz a la API mediante jQuery (AJAX).

Para mantener el proyecto simple y alineado con el entorno, se servirá el frontend desde Spring Boot en:

`src/main/resources/static/`

1.11.1 Estructura de archivos recomendada

Se propone separar HTML, CSS y JavaScript:

```
1 src/main/resources/static/  
2   index.html  
3   css/  
4     app.css  
5   js/  
6     app.js
```

- `index.html` contendrá el marcado y los contenedores de la interfaz.
- `app.css` incluirá pequeños ajustes de estilo (si son necesarios).
- `app.js` contendrá la lógica de interfaz y, posteriormente, las llamadas AJAX.

1.11.2 Diseño de la interfaz

La aplicación tendrá tres áreas funcionales:

1. **Instalaciones:** listado y alta rápida.
2. **Usuarios:** listado y alta rápida.
3. **Reservas:** listado con filtros y formulario de alta.

Se implementará con componentes Bootstrap estándar:

- `navbar` para navegación.
- `card` para secciones.
- `table` para listados.
- `form` para altas y filtros.
- `modal` (opcional) para edición o confirmación.
- `alert` para mensajes de éxito/error.

1.11.3 `index.html` (estructura base)

A continuación se define una versión inicial del `index.html` con Bootstrap y contenedores listos para enlazar con JavaScript. Esta versión aún no realiza peticiones a la API; únicamente prepara la estructura.

```
1 <!doctype html>
2 <html lang="es">
3 <head>
4   <meta charset="utf-8" />
5   <meta name="viewport" content="width=device-width, initial-scale=1" /
6   <title>Reservas de Pistas</title>
7
8   <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/
9     bootstrap.min.css" rel="stylesheet">
10  <link rel="stylesheet" href="css/app.css">
11 </head>
12 <body class="bg-light">
13
14 <nav class="navbar navbar-expand-lg navbar-dark bg-dark">
15   <div class="container">
16     <a class="navbar-brand" href="#">Reservas de Pistas</a>
17     <button class="navbar-toggler" type="button" data-bs-toggle="
18       collapse" data-bs-target="#nav">
19       <span class="navbar-toggler-icon"></span>
20     </button>
21
22     <div class="collapse navbar-collapse" id="nav">
23       <ul class="navbar-nav ms-auto">
24         <li class="nav-item"><a class="nav-link" href="#instalaciones">
25           Instalaciones</a></li>
26         <li class="nav-item"><a class="nav-link" href="#usuarios">
27           Usuarios</a></li>
28         <li class="nav-item"><a class="nav-link" href="#reservas">
29           Reservas</a></li>
30       </ul>
31     </div>
32   </div>
33 </nav>
```

```

29 </nav>
30
31 <main class="container py-4">
32
33   <!-- Área de alertas -->
34   <div id="alerta" class="alert d-none" role="alert"></div>
35
36   <!-- Instalaciones -->
37   <section id="instalaciones" class="mb-4">
38     <div class="card shadow-sm">
39       <div class="card-header fw-semibold">Instalaciones</div>
40       <div class="card-body">
41
42         <form id="formInstalacion" class="row g-2 align-items-end">
43           <div class="col-md-4">
44             <label class="form-label">Nombre</label>
45             <input id="instNombre" class="form-control" required>
46           </div>
47           <div class="col-md-4">
48             <label class="form-label">Dirección</label>
49             <input id="instDireccion" class="form-control" required>
50           </div>
51           <div class="col-md-3">
52             <label class="form-label">Ciudad</label>
53             <input id="instCiudad" class="form-control" required>
54           </div>
55           <div class="col-md-1 d-grid">
56             <button class="btn btn-primary" type="submit">Añadir</button>
57           </div>
58         </form>
59
60         <hr>
61
62         <div class="table-responsive">
63           <table class="table table-hover align-middle">
64             <thead>
65               <tr>
66                 <th>Nombre</th>
67                 <th>Dirección</th>
68                 <th>Ciudad</th>
69                 <th class="text-end">Acciones</th>
70               </tr>
71             </thead>
72             <tbody id="tablaInstalaciones"></tbody>
73           </table>
74         </div>
75
76       </div>
77     </div>
78   </section>

```

```

79
80 <!-- Usuarios -->
81 <section id="usuarios" class="mb-4">
82   <div class="card shadow-sm">
83     <div class="card-header fw-semibold">Usuarios</div>
84     <div class="card-body">
85
86       <form id="formUsuario" class="row g-2 align-items-end">
87         <div class="col-md-5">
88           <label class="form-label">Nombre</label>
89           <input id="userNombre" class="form-control" required>
90         </div>
91         <div class="col-md-5">
92           <label class="form-label">Email</label>
93           <input id="userEmail" type="email" class="form-control"
94             required>
95         </div>
96         <div class="col-md-2 d-grid">
97           <button class="btn btn-primary" type="submit">Añadir</
98             button>
99         </div>
100       </form>
101
102       <hr>
103
104       <div class="table-responsive">
105         <table class="table table-hover align-middle">
106           <thead>
107             <tr>
108               <th>Nombre</th>
109               <th>Email</th>
110               <th class="text-end">Acciones</th>
111             </tr>
112           </thead>
113           <tbody id="tablaUsuarios"></tbody>
114         </table>
115       </div>
116     </div>
117   </section>
118
119 <!-- Reservas -->
120 <section id="reservas">
121   <div class="card shadow-sm">
122     <div class="card-header fw-semibold">Reservas</div>
123     <div class="card-body">
124
125       <!-- Filtros -->
126       <form id="formFiltroReservas" class="row g-2 align-items-end">
127         <div class="col-md-4">

```

```

128         <label class="form-label">Usuario</label>
129         <select id="filtroUsuario" class="form-select">
130             <option value="">Todos</option>
131         </select>
132     </div>
133     <div class="col-md-4">
134         <label class="form-label">Instalación</label>
135         <select id="filtroInstalacion" class="form-select">
136             <option value="">Todas</option>
137         </select>
138     </div>
139     <div class="col-md-3">
140         <label class="form-label">Día</label>
141         <input id="filtroDia" type="date" class="form-control">
142     </div>
143     <div class="col-md-1 d-grid">
144         <button class="btn btn-outline-secondary" type="submit">
145             Filtrar</button>
146     </div>
147 </form>
148 <hr>
149
150 <!-- Alta de reserva -->
151 <form id="formReserva" class="row g-2 align-items-end">
152     <div class="col-md-4">
153         <label class="form-label">Usuario</label>
154         <select id="resUsuario" class="form-select" required></
155             select>
156     </div>
157     <div class="col-md-4">
158         <label class="form-label">Instalación</label>
159         <select id="resInstalacion" class="form-select" required></
160             select>
161     </div>
162     <div class="col-md-2">
163         <label class="form-label">Día</label>
164         <input id="resDia" type="date" class="form-control"
165             required>
166     </div>
167     <div class="col-md-1">
168         <label class="form-label">Inicio</label>
169         <input id="resHoraInicio" type="time" class="form-control"
170             required>
171     </div>
172     <div class="col-md-1">
173         <label class="form-label">Fin</label>
174         <input id="resHoraFin" type="time" class="form-control"
175             required>
176     </div>
177 </form>
178 <div class="col-12 d-grid mt-2">

```

```

173         <button class="btn btn-success" type="submit">Crear reserva
174         </button>
175     </div>
176 </form>
177 <hr>
178
179 <div class="table-responsive">
180     <table class="table table-hover align-middle">
181         <thead>
182             <tr>
183                 <th>Día</th>
184                 <th>Horario</th>
185                 <th>Instalación</th>
186                 <th>Usuario</th>
187                 <th class="text-end">Acciones</th>
188             </tr>
189         </thead>
190         <tbody id="tablaReservas"></tbody>
191     </table>
192 </div>
193
194 </div>
195 </div>
196 </section>
197
198 </main>
199
200 <script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/
201     bootstrap.bundle.min.js"></script>
202 <script src="js/app.js"></script>
203 </body>
204 </html>

```

1.11.4 css/app.css (opcional)

En general, Bootstrap es suficiente. Se recomienda evitar estilos excesivos y limitarse a pequeños ajustes:

```

1  /* app.css */
2  body {
3      min-height: 100vh;
4  }
5
6  #alerta {
7      position: sticky;
8      top: 1rem;
9      z-index: 1020;
10 }

```


1.11.5 Plantillas de tabla (contrato de renderizado)

Aunque el renderizado se implementará en el siguiente apartado con jQuery, conviene fijar desde ya qué se mostrará:

1.11.5.1 Tabla de instalaciones

- Nombre
- Dirección
- Ciudad
- Botón eliminar

1.11.5.2 Tabla de usuarios

- Nombre
- Email
- Botón eliminar

1.11.5.3 Tabla de reservas

- Día (`horario.dia`)
- Franja (`horaInicio` - `horaFin`)
- Instalación (desde snapshot embebido: `horario.instalacionSnapshot.nombre`)
- Usuario (inicialmente se mostrará `usuarioId`; se podrá resolver a nombre en el frontend si se carga la lista de usuarios)
- Botón eliminar

1.11.6 Objetivo técnico del apartado

Al finalizar este apartado se dispone de:

- Una interfaz web estática con Bootstrap.
- Formularios y tablas con identificadores claros (`id`) para conectar con JavaScript.
- Separación correcta de recursos (instalaciones, usuarios, reservas).
- Preparación para implementar el consumo real de la API mediante AJAX en el siguiente apartado.

En el siguiente apartado se implementará “**Consumo de API desde JS (jQuery)**”, que añadirá:

- Carga inicial de datos (GET).

- Altas (POST) desde formularios.
- Eliminación (DELETE).
- Filtrado de reservas usando parámetros de query.
- Gestión de errores utilizando el formato `ApiError` definido en el backend.

1.11.7 Consumo de API desde JS (jQuery)

En este apartado se implementa la lógica del cliente para consumir la API REST mediante **jQuery** y peticiones AJAX. El objetivo es que el frontend:

- Cargue los datos iniciales (instalaciones, usuarios, reservas).
- Permita crear instalaciones, usuarios y reservas desde formularios.
- Permita eliminar registros.
- Permita filtrar reservas mediante parámetros de consulta.
- Gestione correctamente errores de validación y conflictos (por ejemplo, solapes de reservas) utilizando el formato `ApiError` devuelto por la API.

Se asume la estructura creada en el apartado anterior:

- `src/main/resources/static/index.html`
- `src/main/resources/static/js/app.js`
- `src/main/resources/static/css/app.css`

Este apartado implementa `js/app.js`.

1.11.7.1 Convenciones y endpoints usados Se utilizarán los endpoints definidos previamente:

- Instalaciones: `/api/instalaciones`
- Usuarios: `/api/usuarios`
- Reservas: `/api/reservas`

Las respuestas de error seguirán el modelo:

```
1 {
2   "timestamp": "...",
3   "status": 400,
4   "error": "VALIDATION",
5   "message": "Datos no válidos",
6   "details": ["campo: mensaje", "..."]
7 }
```

1.11.7.2 Implementación del cliente: `static/js/app.js`

```
1 /* global $ */
2
3 const API = {
4   instalaciones: "/api/instalaciones",
5   usuarios: "/api/usuarios",
6   reservas: "/api/reservas"
7 };
8
```

```
9 $(document).ready(function () {
10     wireEvents();
11     cargarTodo();
12 });
13
14 /* =====
15     Eventos de formularios
16     ===== */
17
18 function wireEvents() {
19     $("#formInstalacion").on("submit", function (e) {
20         e.preventDefault();
21         crearInstalacion();
22     });
23
24     $("#formUsuario").on("submit", function (e) {
25         e.preventDefault();
26         crearUsuario();
27     });
28
29     $("#formReserva").on("submit", function (e) {
30         e.preventDefault();
31         crearReserva();
32     });
33
34     $("#formFiltroReservas").on("submit", function (e) {
35         e.preventDefault();
36         cargarReservasConFiltros();
37     });
38 }
39
40 /* =====
41     Alertas y utilidades
42     ===== */
43
44 function showAlert(type, msg) {
45     $("#alerta")
46         .removeClass("d-none alert-success alert-danger alert-warning alert
47             -info")
48         .addClass("alert-" + type)
49         .text(msg);
50     setTimeout(() => $("#alerta").addClass("d-none"), 3000);
51 }
52
53 function parseApiError(xhr, fallbackMsg) {
54     const r = xhr.responseJSON;
55     if (!r) return fallbackMsg;
56
57     if (Array.isArray(r.details) && r.details.length > 0) {
58         return `${r.message}: ${r.details.join(" | ")}`;
```

```
59     }
60     return r.message || fallbackMsg;
61 }
62
63 function escapeHtml(s) {
64     return String(s)
65         .replaceAll("&", "&amp;")
66         .replaceAll("<", "&lt;")
67         .replaceAll(">", "&gt;");
68 }
69
70 /* =====
71     Carga inicial
72     ===== */
73
74 function cargarTodo() {
75     $.when(cargarInstalaciones(), cargarUsuarios())
76         .done(function () {
77             cargarReservas();
78         })
79         .fail(function () {
80             showAlert("danger", "Error cargando datos iniciales");
81         });
82 }
83
84 /* =====
85     Instalaciones
86     ===== */
87
88 function cargarInstalaciones() {
89     return $.getJSON(API.instalaciones)
90         .done(function (data) {
91             renderInstalaciones(data);
92             rellenarSelectInstalaciones(data);
93         })
94         .fail(function (xhr) {
95             showAlert("danger", parseApiError(xhr, "Error cargando
96                 instalaciones"));
97         });
98 }
99
100 function renderInstalaciones(instalaciones) {
101     const rows = (instalaciones || []).map(function (i) {
102         return `
103             <tr>
104                 <td>${escapeHtml(i.nombre)}</td>
105                 <td>${escapeHtml(i.direccion)}</td>
106                 <td>${escapeHtml(i.ciudad)}</td>
107                 <td class="text-end">
108                     <button class="btn btn-sm btn-outline-danger" data-action="
109                         del-inst" data-id="${i.id}">
```

```
108         Eliminar
109         </button>
110     </td>
111 </tr>
112 `;
113 }).join("");
114
115 $("#tablaInstalaciones").html(rows || `<tr><td colspan="4" class="
    text-center text-muted">Sin datos</td></tr>`);
116
117 // Delegación de eventos para botones generados dinámicamente
118 $("#tablaInstalaciones button[data-action='del-inst']").off("click").
    on("click", function () {
119     const id = $(this).data("id");
120     eliminarInstalacion(id);
121 });
122 }
123
124 function rellenarSelectInstalaciones(instalaciones) {
125     const opts = (instalaciones || []).map(i =>
126         `<option value="${i.id}">${escapeHtml(i.nombre)} (${escapeHtml(i.
            ciudad)})</option>`
127     ).join("");
128
129     // Selects: filtros y alta
130     $("#filtroInstalacion").html(`<option value="">Todas</option>${opts}
        `);
131     $("#resInstalacion").html(`<option value="" disabled selected>
        Seleccione...</option>${opts}`);
132 }
133
134 function crearInstalacion() {
135     const payload = {
136         nombre: $("#instNombre").val().trim(),
137         direccion: $("#instDireccion").val().trim(),
138         ciudad: $("#instCiudad").val().trim()
139     };
140
141     $.ajax({
142         url: API.instalaciones,
143         method: "POST",
144         contentType: "application/json",
145         data: JSON.stringify(payload)
146     })
147     .done(function () {
148         showAlert("success", "Instalación creada");
149         $("#formInstalacion")[0].reset();
150         cargarInstalaciones().done(cargarReservasConFiltros);
151     })
152     .fail(function (xhr) {
```

```
153     showAlert("danger", parseApiError(xhr, "Error creando instalación
154         "));
155 }
156
157 function eliminarInstalacion(id) {
158     if (!confirm("? Eliminar la instalación?")) return;
159
160     $.ajax({
161         url: `${API.instalaciones}/${id}`,
162         method: "DELETE"
163     })
164     .done(function () {
165         showAlert("success", "Instalación eliminada");
166         cargarInstalaciones().done(cargarReservasConFiltros);
167     })
168     .fail(function (xhr) {
169         showAlert("danger", parseApiError(xhr, "Error eliminando
170             instalación"));
171     });
172 }
173
174 /* =====
175    Usuarios
176    ===== */
177
178 function cargarUsuarios() {
179     return $.getJSON(API.usuarios)
180     .done(function (data) {
181         renderUsuarios(data);
182         rellenarSelectUsuarios(data);
183     })
184     .fail(function (xhr) {
185         showAlert("danger", parseApiError(xhr, "Error cargando usuarios")
186     );
187     });
188 }
189
190 function renderUsuarios(usuarios) {
191     const rows = (usuarios || []).map(function (u) {
192         return `
193         <tr>
194             <td>${escapeHtml(u.nombre)}</td>
195             <td>${escapeHtml(u.email)}</td>
196             <td class="text-end">
197                 <button class="btn btn-sm btn-outline-danger" data-action="
198                     del-user" data-id="${u.id}">
199                     Eliminar
200                 </button>
201             </td>
202         </tr>`
203     });
204     return rows.join("");
205 }
```

```
200     `;
201   }).join("");
202
203   $("#tablaUsuarios").html(rows || `<tr><td colspan="3" class="text-
204     center text-muted">Sin datos</td></tr>`);
205
206   $("#tablaUsuarios button[data-action='del-user']").off("click").on("
207     click", function () {
208     const id = $(this).data("id");
209     eliminarUsuario(id);
210   });
211 }
212
213 function rellenarSelectUsuarios(usuarios) {
214   const opts = (usuarios || []).map(u =>
215     `<option value="${u.id}">${escapeHtml(u.nombre)} (${escapeHtml(u.
216       email)})</option>`
217   ).join("");
218
219   $("#filtroUsuario").html(`<option value="">Todos</option>${opts}`);
220   $("#resUsuario").html(`<option value="" disabled selected>Seleccione
221     ...</option>${opts}`);
222 }
223
224 function crearUsuario() {
225   const payload = {
226     nombre: $("#userNombre").val().trim(),
227     email: $("#userEmail").val().trim()
228   };
229
230   $.ajax({
231     url: API.usuarios,
232     method: "POST",
233     contentType: "application/json",
234     data: JSON.stringify(payload)
235   })
236     .done(function () {
237       showAlert("success", "Usuario creado");
238       $("#formUsuario")[0].reset();
239       cargarUsuarios().done(cargarReservasConFiltros);
240     })
241     .fail(function (xhr) {
242       showAlert("danger", parseApiError(xhr, "Error creando usuario"));
243     });
244 }
245
246 function eliminarUsuario(id) {
247   if (!confirm("? Eliminar el usuario?")) return;
248
249   $.ajax({
250     url: `${API.usuarios}/${id}`,
```



```
247     method: "DELETE"
248   })
249   .done(function () {
250     showAlert("success", "Usuario eliminado");
251     cargarUsuarios().done(cargarReservasConFiltros);
252   })
253   .fail(function (xhr) {
254     showAlert("danger", parseApiError(xhr, "Error eliminando usuario"
255     ));
256   });
257 }
258 /* =====
259     Reservas
260     ===== */
261
262 function cargarReservas() {
263   return $.getJSON(API.reservas)
264     .done(function (data) {
265       renderReservas(data);
266     })
267     .fail(function (xhr) {
268       showAlert("danger", parseApiError(xhr, "Error cargando reservas")
269       );
270     });
271 }
272
273 function cargarReservasConFiltros() {
274   const usuarioId = $("#filtroUsuario").val();
275   const instalacionId = $("#filtroInstalacion").val();
276   const dia = $("#filtroDia").val();
277
278   const params = {};
279   if (usuarioId) params.usuarioId = usuarioId;
280   if (instalacionId) params.instalacionId = instalacionId;
281   if (dia) params.dia = dia;
282
283   const query = $.param(params);
284   const url = query ? `${API.reservas}?${query}` : API.reservas;
285
286   return $.getJSON(url)
287     .done(function (data) {
288       renderReservas(data);
289     })
290     .fail(function (xhr) {
291       showAlert("danger", parseApiError(xhr, "Error filtrando reservas"
292       ));
293     });
294 }
295
296 function crearReserva() {
```

```
295     const payload = {
296       usuarioId: $("#resUsuario").val(),
297       instalacionId: $("#resInstalacion").val(),
298       dia: $("#resDia").val(),
299       horaInicio: $("#resHoraInicio").val(),
300       horaFin: $("#resHoraFin").val()
301     };
302
303     $.ajax({
304       url: API.reservas,
305       method: "POST",
306       contentType: "application/json",
307       data: JSON.stringify(payload)
308     })
309     .done(function () {
310       showAlert("success", "Reserva creada");
311       $("#formReserva")[0].reset();
312       cargarReservasConFiltros();
313     })
314     .fail(function (xhr) {
315       // 409 típico por solape, 400 por validación, 404 por usuario/
316       // instalación inexistente
317       showAlert("danger", parseApiError(xhr, "Error creando reserva"));
318     });
319
320     function eliminarReserva(id) {
321       if (!confirm("? Eliminar la reserva?")) return;
322
323       $.ajax({
324         url: `${API.reservas}/${id}`,
325         method: "DELETE"
326       })
327       .done(function () {
328         showAlert("success", "Reserva eliminada");
329         cargarReservasConFiltros();
330       })
331       .fail(function (xhr) {
332         showAlert("danger", parseApiError(xhr, "Error eliminando reserva"));
333       });
334     }
335
336     function renderReservas(reservas) {
337       const usuariosMap = construirUsuariosMap();
338       const rows = (reservas || []).map(function (r) {
339         const h = r.horario || {};
340         const snap = h.instalacionSnapshot || {};
341         const usuarioNombre = usuariosMap.get(r.usuarioId) || r.usuarioId;
342
343         const dia = h.dia || "";
```

```

344     const tramo = `${h.horaInicio || ""} - ${h.horaFin || ""}`;
345     const instalacion = snap.nombre ? `${snap.nombre} (${snap.ciudad || ""})` : (snap.instalacionId || "");
346
347     return `
348         <tr>
349             <td>${escapeHtml(dia)}</td>
350             <td>${escapeHtml(tramo)}</td>
351             <td>${escapeHtml(instalacion)}</td>
352             <td>${escapeHtml(usuarioNombre)}</td>
353             <td class="text-end">
354                 <button class="btn btn-sm btn-outline-danger" data-action="
355                     del-res" data-id="${r.id}">
356                     Eliminar
357                 </button>
358             </td>
359         </tr>
360     `;
361     }).join("");
362     $("#tablaReservas").html(rows || `<tr><td colspan="5" class="text-center text-muted">Sin datos</td></tr>`);
363
364     $("#tablaReservas button[data-action='del-res']").off("click").on("
365         click", function () {
366         const id = $(this).data("id");
367         eliminarReserva(id);
368     });
369
370     /* =====
371     Mapas auxiliares
372     ===== */
373
374     function construirUsuariosMap() {
375         // Construye un mapa id -> nombre leyendo la tabla ya cargada.
376         // Alternativa: guardar el último listado en una variable global.
377         const map = new Map();
378
379         // Se intenta construir desde el select de reservas (que contiene
380         // nombre + email).
381         $("#resUsuario option").each(function () {
382             const val = $(this).attr("value");
383             const txt = $(this).text();
384             if (val) map.set(val, txt);
385         });
386
387         return map;

```

1.11.7.2.1 Explicación técnica de la implementación Carga inicial (flujo)

1. Se cargan instalaciones y usuarios (necesarios para los `select`).
2. Una vez cargados, se cargan reservas.

Esto evita que el formulario de alta de reservas aparezca sin opciones.

Gestión de errores con `ApiError`

La función `parseApiError` intenta extraer:

- `message`
- `details[]`

Así, los errores de validación se muestran con detalle, y los conflictos (409) se reportan de forma clara.

Consultas con filtros

El filtrado se realiza construyendo la query string con `$.param()` para:

- `usuarioId`
- `instalacionId`
- `dia`

1.11.7.2.2 Resultado de este apartado Al finalizar este apartado el frontend queda plenamente funcional:

- CRUD básico de instalaciones (crear/listar/eliminar).
- CRUD básico de usuarios (crear/listar/eliminar).
- CRUD básico de reservas (crear/listar/eliminar).
- Filtros sobre reservas.
- Mensajes de error consistentes.

En el siguiente apartado se desarrollará **“Caso práctico completo (paso a paso)”**, donde se integrará todo el flujo en orden: creación de datos base, creación de reservas, pruebas con Postman, verificación desde la interfaz web y comprobación de las reglas de negocio (incluyendo el conflicto por solape).

2 Bibliografía

- Documentación oficial de MongoDB: (<https://www.mongodb.com/docs/>).